

République Tunisienne
Ministère de l'Enseignement Supérieur
et de la Recherche Scientifique

Université de Sfax

École nationale d'électronique
et des télécommunications de Sfax



Ingénieur en :
Ingénierie des données
et systèmes décisionnels

Option :
Informatique Décisionnelle

N° d'ordre : IDS-D-3-2025-025

MEMOIRE

présenté à

**L'École Nationale d'Électronique et des
Télécommunications de Sfax**

en vue de l'obtention du

**Diplôme National d'Ingénieur en :
Ingénierie des données et systèmes décisionnels**

**Option :
Informatique Décisionnelle**

par

Amina CHAABEN

**Génération automatique de scripts de test à partir de
scénarios Gherkin et exécution autonome via UI**

Soutenu le 21/06/2025, devant la commission d'examen :

Mme. Molka REKIK	Président
M. Tarak OUNI	Rapporteur
M. Boukthir HADDAR	Encadrant Académique
M. Mohamed Amine AISSAOUI	Encadrant Professionnel



DEDICATION

À mes parents, pour leur amour inconditionnel et leur soutien indéfectible tout au long de
mes études.

À mon frère et ma sœur, pour leur encouragement et leur bonne humeur constante, qui m'ont
permis de garder confiance dans les moments difficiles.

À mes amis proches, pour leurs conseils avisés et leur présence précieuse lors des relectures
et discussions autour de ce projet

À vous tous,

Je dédie ce travail.

Amina CHAABEN



REMERCIEMENT

Je tiens à exprimer ma profonde gratitude à M. Boukthir HADDAR, mon encadrant académique à l'ENET'COM, pour ses conseils éclairés et sa disponibilité tout au long de la réalisation de ce projet.

Je remercie également M. Mohamed Amine AISSAOUI et Mme Farah OUESLATI, mes encadrants professionnels chez Talan, pour leur expertise, leur accompagnement constant et leur soutien technique durant ce projet.

J'adresse mes remerciements à Mme Molka REKIK, présidente de la commission d'examen, et à M. Tarak OUNI, rapporteur, pour le temps et l'attention qu'ils ont consacrés à l'évaluation de ce travail.

Merci enfin à l'ensemble des enseignants et du personnel de l'ENET'COM ainsi qu'à toute l'équipe de Talan pour leur encadrement, leur soutien pédagogique et organisationnel, qui ont grandement contribué à la bonne conduite de ce projet.



TABLE DES MATIÈRES

LISTE DES FIGURES	vii
LISTE DES TABLEAUX	viii
LISTE DES ABRÉVIATIONS	ix
INTRODUCTION GÉNÉRALE	1
1 Cadre général du projet	2
1.1 INTRODUCTION	3
1.2 Présentation de l'entreprise d'accueil	3
1.2.1 Présentation de Talan	3
1.2.2 Les expertises de Talan	3
1.3 Cadre du projet	4
1.3.1 Problématique	5
1.3.2 Etude de l'existant	5
1.3.3 Solution Proposée	7
1.4 Méthodologie de travail	8
1.4.1 Méthodologie Agile	8
1.4.2 Méthodologie SCRUM	8
1.4.3 Les rôles dans SCRUM	9
1.4.4 Planification des sprint	10
1.5 Outil de gestion du projet	11
1.6 CONCLUSION	12
2 Étude conceptuelle et théorique	13
2.1 INTRODUCTION	14
2.2 Compréhension du métier	14
2.2.1 User Stories	14
2.2.2 Langage Gherkin	15

2.2.3	Documents de spécification	15
2.2.4	Cycle de vie du testing logiciel	15
2.3	Les Besoins fonctionnels et non fonctionnels	16
2.3.1	Analyse et spécifications des besoins	16
2.3.2	Besoins fonctionnels	17
2.3.2.1	Génération de projets de tests	17
2.3.2.2	Exécution des tests automatisés via l’UI	17
2.3.3	Besoins non fonctionnels	17
2.3.3.1	Génération de projets de tests	17
2.3.3.2	Exécution de tests automatisés via l’UI	18
2.4	Fondements théoriques	18
2.4.1	Agents intelligents	18
2.4.2	Théorie des agents autonomes	18
2.4.3	Systèmes multi-agents	19
2.4.4	Modèles de Langages de Grandes Tailles (LLM)	19
2.4.5	Traitement du langage naturel (NLP)	20
2.4.6	IA Générative	20
2.4.7	Vision par Ordinateur	21
2.5	Plan de travail	21
2.6	Vision globale du projet	22
2.7	CONCLUSION	23
3	Génération de projet de tests automatisés	24
3.1	INTRODUCTION	25
3.2	Architecture du projet	25
3.3	Fonctionnalités	28
3.4	Outils	30
3.5	Benchmark des outils d’analyse visuelle	30
3.5.1	OmniParser V1 (Microsoft)	31
3.5.2	Pix2Struct (Google)	31
3.5.3	Choix d’OmniParser	32
3.6	Benchmark des LLM testés	33
3.7	Choix des LLM	37
3.8	Gestion de la robustesse des sélecteurs	38
3.9	Structure du code généré	39
3.9.1	Modèle de structure adopté	40

3.9.1.1	Script de test fonctionnel	40
3.9.1.2	Assertions explicites	40
3.9.1.3	Utilisation de waits explicites	40
3.9.2	Réutilisabilité du code généré	40
3.9.3	Exemple annoté d'un code test généré	41
3.9.4	Synthèse	41
3.10	CONCLUSION	42
4	Exécution de tests automatisés basés sur l'UI	43
4.1	INTRODUCTION	44
4.2	Architecture du projet	44
4.3	Fonctionnalités	46
4.4	Outils	47
4.5	Benchmark des outils d'exécution automatisée	47
4.5.1	Comparaison fonctionnelle	48
4.5.2	Synthèse des atouts	48
4.5.3	Choix de Browser-Use	49
4.6	Benchmark des LLM testés	50
4.7	Normalisation des scénarios Gherkin	51
4.7.1	Prétraitement syntaxique	51
4.7.2	Utilisation de modèles d'étapes structurés	51
4.7.3	Agent de reformulation et d'enrichissement	52
4.8	Techniques de <i>Prompt Engineering</i> utilisées	52
4.9	CONCLUSION	53
5	Résultats et Interface	54
5.1	Introduction	55
5.2	Évaluation du module de génération de cas de test	55
5.2.1	Temps de génération	55
5.2.2	Robustesse des sélecteurs	55
5.2.3	Alignement scénario–interface	56
5.2.4	Interface de génération	56
5.3	Évaluation du module d'exécution des tests	60
5.3.1	Temps d'exécution	60
5.3.2	Taux de réussite	60
5.3.3	Rapports d'exécution	60

5.3.4	Interface de suivi d'exécution	61
5.4	Apport de l'interface utilisateur ReactJS	63
5.4.1	Objectifs de l'interface	63
5.4.2	Architecture de communication	63
5.4.3	Expérience utilisateur	63
5.5	Conclusion	64
CONCLUSION GÉNÉRALE		65
BIBLIOGRAPHIE		66

LISTE DES FIGURES

1.1	Logo de l'entreprise Talan Tunisie Consulting	4
1.2	Cycle de travail selon la méthodologie Scrum	9
1.3	Tableau Trello utilisé pour la gestion des tâches par sprint	10
2.1	Cycle de vie du testing logiciel (STLC)	16
2.2	Diagramme de Gantt du projet	21
2.3	Vision globale du projet	23
3.1	Architecture de génération de projets de tests automatisés	25
3.2	Image capturée de l'interface utilisateur	26
3.3	Image annotée avec les éléments UI détectés	26
3.4	Exemple de prompt <i>zero-shot</i> pour la génération de code Selenium à partir d'un scénario Gherkin.	34
3.5	Exemple de prompt <i>one-shot</i> avec un scénario Gherkin et son code Selenium avant la requête cible.	35
3.6	Exemple de prompt <i>two-shot</i> montrant deux scénarios complets avant le prompt cible.	36
3.7	Tableau Trello utilisé pour la gestion des tâches par sprint	41
4.1	Architecture d'exécution de tests automatisés basés sur l'interface utilisateur	44
5.1	Accueil de l'interface – Importation des fichiers .feature	56
5.2	Accueil de l'interface – Importation d'un scénario Gherkin	57
5.3	Visualisation des sélecteurs détectés	57
5.4	Aperçu du code de test généré	58
5.5	Téléchargement du projet de test	59
5.6	Accueil de la plateforme d'exécution	61
5.7	Exécution parallèle de cas de test	61
5.8	Étapes de raisonnement de l'agent d'exécution	62
5.9	Vue d'ensemble du rapport Allure	62
5.10	Détail du statut d'exécution par cas de test	62



LISTE DES TABLEAUX

1.1	Outils utilisés pour la gestion du projet	11
3.1	Outils utilisés pour la capture, l'analyse, la détection visuelle et la gestion des cas de tests	30
3.2	Évaluation comparative de plusieurs modèles LLM sur différentes tâches liées à la génération de tests automatisés.	37
4.1	Outils principaux utilisés pour l'exécution, la gestion et le reporting des cas de tests	47
4.2	Comparaison entre Browser-Use et CAMEL-AI	48
4.3	Comparaison des performances des modèles sur divers benchmarks	50



LISTE DES ABRÉVIATIONS

AI Artificial Intelligence (Intelligence Artificielle)

API Application Programming Interface (Interface de Programmation d'Application)

CI/CD Continuous Integration / Continuous Deployment (Intégration Continue / Déploiement Continu)

CSS Cascading Style Sheets (Feuilles de style en cascade)

DOM Document Object Model (Modèle Objet du Document)

Gherkin Langage structuré de description de scénarios

GPU Graphics Processing Unit (Processeur Graphique)

HTML HyperText Markup Language (Langage de balisage hypertexte)

JSON JavaScript Object Notation

LLM Large Language Model (Modèle de Langage de Grande Taille)

NLP Natural Language Processing (Traitement du Langage Naturel)

POM Page Object Model (Modèle Objet de Page)

QA Quality Assurance (Assurance Qualité)

ReactJS Bibliothèque JavaScript pour la création d'interfaces utilisateur

STLC Software Testing Life Cycle (Cycle de vie du test logiciel)

UI User Interface (Interface Utilisateur)

URL Uniform Resource Locator (Localisateur Uniforme de Ressource)

XPath XML Path Language (Langage de chemin pour XML)



INTRODUCTION GÉNÉRALE

L'automatisation des tests logiciels est devenue un enjeu central dans les processus de développement modernes. Avec l'essor des architectures complexes et des exigences de livraison rapide, les équipes de qualité doivent sans cesse adapter leurs méthodes pour garantir la fiabilité et la robustesse des systèmes déployés.

Cependant, malgré la diversité des outils disponibles, plusieurs freins persistent : scripts peu maintenables, faible flexibilité face aux changements d'interface, exécution difficile à grande échelle ou encore analyse lente des échecs. Ces défis compromettent la rentabilité et l'efficacité de l'automatisation dans les projets à fort volume.

Dans ce contexte, nous avons développé une solution hybride de génération et d'exécution automatisées de cas de test. Elle repose sur l'analyse sémantique de scénarios Gherkin couplée à une compréhension de l'interface utilisateur (DOM + image), avec exécution via navigateur, sans recours au code source.

Notre contribution s'articule autour de deux modules : un moteur de génération de tests robustes et maintenables, ainsi qu'un exécuteur intelligent piloté par UI. Une interface ReactJS complète ce système pour simplifier son adoption. L'ensemble a été validé par des benchmarks et expérimentations réels.

Le reste de ce rapport est structuré comme suit. Le chapitre 1 présente le cadre du projet, la problématique et les outils analysés. Le chapitre 2 développe les bases conceptuelles et les choix technologiques. Le chapitre 3 décrit la mise en œuvre de la génération de tests. Le chapitre 4 traite de l'exécution intelligente. Le chapitre 5 évalue notre solution et présente l'interface utilisateur. Enfin, ce travail se conclut par une synthèse générale et des perspectives.

Cadre général du projet

Sommaire

1.1	INTRODUCTION	3
1.2	Présentation de l'entreprise d'accueil	3
1.2.1	Présentation de Talan	3
1.2.2	Les expertises de Talan	3
1.3	Cadre du projet	4
1.3.1	Problématique	5
1.3.2	Etude de l'existant	5
1.3.3	Solution Proposée	7
1.4	Méthodologie de travail	8
1.4.1	Méthodologie Agile	8
1.4.2	Méthodologie SCRUM	8
1.4.3	Les rôles dans SCRUM	9
1.4.4	Planification des sprint	10
1.5	Outil de gestion du projet	11
1.6	CONCLUSION	12

1.1 INTRODUCTION

Dans ce chapitre, nous présentons l'environnement professionnel du projet, en introduisant l'entreprise d'accueil et ses domaines d'expertise. Nous exposons ensuite le cadre général du projet, en détaillant la problématique à résoudre et les limites des solutions existantes. Une description de la solution proposée ainsi que la méthodologie de travail adoptée concluent ce chapitre.

1.2 Présentation de l'entreprise d'accueil

1.2.1 Présentation de Talan

Talan est un groupe international de conseil et d'expertise technologique, présent dans 21 pays à travers 5 continents, avec plus de 7000 collaborateurs. Depuis plus de 20 ans, le groupe accompagne les grandes entreprises et les institutions publiques dans la conception et la mise en œuvre de leurs projets de transformation digitale.

Grâce à une forte expertise en innovation, en data, en intelligence artificielle et en cybersécurité, Talan intervient comme un partenaire stratégique pour aider ses clients à relever les défis technologiques et sociétaux actuels. Le groupe a réalisé un chiffre d'affaires de 830 millions d'euros en 2024 et vise le cap du milliard d'euros en 2025. Sa stratégie repose sur une innovation responsable, centrée sur l'humain, et sur une croissance durable appuyée par la recherche, les partenariats technologiques et les acquisitions ciblées.

1.2.2 Les expertises de Talan

Talan propose une offre complète d'expertises pour accompagner ses clients dans leurs transformations technologiques, organisationnelles et durables. Son approche repose sur une connaissance approfondie des enjeux métiers et sectoriels, ainsi qu'une maîtrise des technologies les plus avancées.

- **Conseil en stratégie et transformation** : accompagnement des transformations organisationnelles, culturelles et technologiques.
- **Data, IA et cybersécurité** : valorisation de la donnée, automatisation intelligente, IA responsable et sécurisation des systèmes.
- **Cloud & applications** : intégration de solutions logicielles de marché, modernisation des systèmes, migration vers le cloud.
- **Développement durable** : mise en œuvre de solutions à faible empreinte carbone, économie circulaire, mobilité durable.
- **Expertise multisectorielle** : présence forte dans les secteurs de la finance, de l'énergie, de la santé, du transport, du retail, des télécoms, etc.



FIGURE 1.1 – Logo de l'entreprise Talan Tunisie Consulting

La figure 1.1 illustre le logo de l'entreprise Talan, notre organisme d'accueil pour ce projet.

1.3 Cadre du projet

Dans le cadre de ma formation en ingénierie des données à l'ENET'COM, j'ai participé à un projet d'automatisation intelligente des tests logiciels. L'objectif était de générer automatiquement des scripts de test à partir de scénarios en langage Gherkin, compatibles avec différents outils et destinés à une exécution manuelle par les testeurs.

En parallèle, une exécution automatique des tests a été mise en place, basée exclusivement sur l'interface utilisateur.

Ce projet s'inscrit dans une démarche d'innovation visant à renforcer l'efficacité des processus de test à l'aide de l'intelligence artificielle.

1.3.1 Problématique

Malgré les avantages de l'automatisation des tests logiciels, plusieurs limitations persistent, entravant son efficacité dans les projets complexes :

- **Tests longs et coûteux à maintenir** : Les scripts d'automatisation nécessitent un investissement initial important pour leur création et leur maintenance, en particulier lorsqu'ils doivent s'adapter à des environnements évolutifs.
- **Risque d'erreurs dans l'automatisation** : Les outils d'automatisation sont sujets à des erreurs de configuration, de synchronisation ou d'intégration, ce qui peut fausser les résultats des tests.
- **Difficulté d'exécution à grande échelle** : Lorsqu'il s'agit d'exécuter des tests sur de multiples plateformes ou configurations, l'automatisation devient plus complexe et moins flexible.
- **Analyse des échecs lente** : Lorsqu'un test échoue, l'identification des causes profondes peut être laborieuse, surtout sans outils d'analyse adaptés et automatisés.
- **Coût élevé pour une couverture complète** : Automatiser l'ensemble des tests d'un projet peut nécessiter des ressources humaines et techniques considérables, rendant cette approche coûteuse à déployer à grande échelle.

Dès lors, comment surmonter ces défis pour rendre l'automatisation des tests plus flexible, évolutive et rentable, tout en maintenant une exécution efficace et fiable ?

1.3.2 Etude de l'existant

1. Operator (OpenAI)

- **Points forts** : Permet l'automatisation de tâches simples, répétitives et facilement réutilisables.
- **Limitations** : Ne prend pas en charge les tâches complexes, accessible uniquement aux utilisateurs de ChatGPT Pro, et disponible exclusivement aux États-Unis.

2. ChatGPT

- **Points forts :** Capacité fiable à générer du code pertinent et fonctionnel.
- **Limitations :** Son approche trop générique limite son adaptation aux besoins spécifiques du projet.

3. ACCELQ

- **Points forts :** Automatisation sans code, intégration multi-cloud.
- **Limitations :** Personnalisation limitée pour des scénarios complexes.

4. BrowserStack

- **Points forts :** Tests multi-navigateurs et mobiles en temps réel.
- **Limitations :** Contrôle limité sur l'environnement de test.

5. TestGrid

- **Points forts :** Facilité d'utilisation et exécution parallèle des tests.
- **Limitations :** Documentation et options limitées.

6. MagicPod

- **Points forts :** Automatisation sans code avec auto-guérison IA.
- **Limitations :** Intégration limitée avec les pipelines CI/CD.

7. LambdaTest

- **Points forts :** Tests cross-browser avec appareils réels.
- **Limitations :** Performance lente, difficulté d'exécution des tests.

Bien que ces solutions apportent des avantages, elles restent limitées par leur manque de personnalisation, leur coût élevé ou leur faible évolutivité. Notre solution, qui génère des tests indépendants de l'interface utilisateur, surmonte ces limitations en offrant plus de flexibilité et d'évolutivité pour des environnements de test complexes.

1.3.3 Solution Proposée

La solution que nous proposons repose sur deux approches complémentaires pour améliorer l'automatisation des tests logiciels :

1. **Génération de Code de Test Hybride** : La génération du code de test repose sur une approche combinée utilisant à la fois **interface utilisateur (UI)** et la **structure du modèle d'objet de document (DOM)** pour générer des locators robustes et adaptatifs :

- **Approche UI** : Les locators sont générés en identifiant visuellement les éléments sur l'interface, garantissant leur accessibilité dans un contexte réel d'utilisation.
- **Approche DOM** : Parallèlement, la génération des locators utilise la structure DOM, offrant une stabilité accrue, même en cas de modifications de l'interface, ce qui permet d'assurer la pérennité des tests dans le temps.

Cette méthode hybride permet de créer des tests plus fiables et adaptés aux changements d'interface tout en maintenant une forte précision dans la génération des locators.

2. **Exécution Optimisée des Test Cases via l'UI** : En plus de la génération de code de test, notre solution optimise l'exécution des tests en se basant uniquement sur l'**interface utilisateur**. Cette approche permet d'exécuter les tests de manière plus rapide et plus efficace en simulant des interactions directes avec l'interface, comme le ferait un utilisateur final. Cela permet de réduire le temps d'exécution des tests tout en conservant leur pertinence et leur fiabilité. L'exécution basée exclusivement sur l'UI offre une plus grande fidélité aux scénarios réels d'utilisation, sans nécessiter une dépendance sur la structure interne du système ou du code sous-jacent, ce qui augmente la flexibilité du processus de test.

Ces deux approches (génération de code de test hybride et exécution optimisée via l'UI) permettent d'améliorer l'efficacité des tests automatisés, en les rendant plus fiables, plus rapides et mieux adaptés aux environnements en constante évolution.

1.4 Méthodologie de travail

La méthodologie représente un ensemble d'approches et de lignes directrices qui permettent d'organiser un projet tout en respectant les bonnes pratiques. Le choix de la bonne méthodologie est crucial pour obtenir des résultats optimaux et assurer une exécution fluide de chaque phase du projet. Dans cette section, nous détaillerons la méthodologie choisie ainsi que le formalisme appliqué.

1.4.1 Méthodologie Agile

La méthodologie agile est une approche de gestion de projet fondée sur l'adaptabilité, la collaboration et la livraison itérative de valeur. Contrairement aux méthodes classiques en cascade, l'agilité favorise une planification souple, des cycles courts de développement et une implication régulière des parties prenantes. Elle repose sur un ensemble de principes définis dans le Manifeste Agile, qui valorise les individus et leurs interactions, des logiciels opérationnels, la collaboration avec le client et l'acceptation du changement.

Cette approche permet de répondre plus efficacement aux besoins évolutifs d'un projet, tout en assurant un haut niveau de qualité à chaque étape du processus de développement.

1.4.2 Méthodologie SCRUM

La méthodologie Scrum est un cadre de gestion de projet agile qui se distingue par sa flexibilité et son approche itérative. Contrairement aux méthodes traditionnelles, Scrum se concentre sur des cycles de travail courts, appelés "sprints", permettant ainsi une évaluation continue des progrès et des ajustements en temps réel. L'objectif est de livrer progressivement des améliorations fonctionnelles du produit, en tenant compte des retours des parties prenantes à chaque itération.

Les pratiques Scrum reposent sur des événements clés, tels que les réunions quotidiennes (daily standups), les revues de sprint et les rétrospectives, qui favorisent la communication, l'amélioration continue et l'adaptabilité face aux changements. En adoptant Scrum, les équipes peuvent ainsi s'assurer que le projet évolue de manière cohérente, tout en restant réactives aux besoins changeants des clients et des parties prenantes.

1.4.3 Les rôles dans SCRUM

Trois rôles clés assurent le bon fonctionnement de Scrum :

- **Product Owner** : il porte la vision du produit, fixe les priorités et veille à la pertinence des fonctionnalités développées.
- **Scrum Master** : il accompagne l'équipe dans l'application de Scrum, facilite les échanges et résout les éventuels blocages.
- **Équipe de développement** : autonome et pluridisciplinaire, elle est chargée de transformer les idées en livrables concrets.

Cette organisation favorise l'efficacité, la responsabilité partagée et une grande réactivité face aux imprévus.

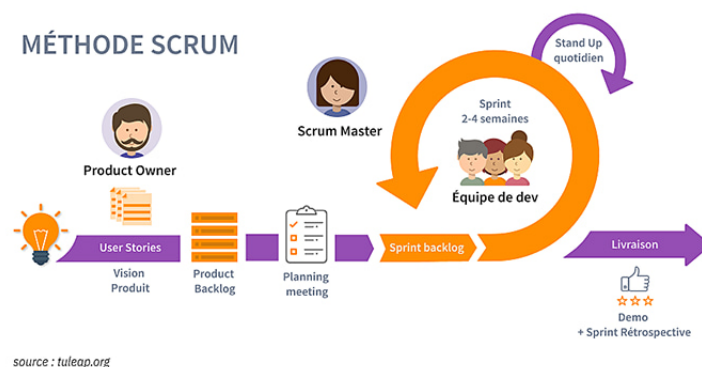


FIGURE 1.2 – Cycle de travail selon la méthodologie Scrum

La figure 1.2 présente l'organisation du projet selon les principes agiles de la méthode Scrum.

1.4.4 Planification des sprint

Dans le cadre de la gestion du projet, nous avons opté pour une planification par sprints afin de structurer efficacement le travail et d'en assurer le suivi régulier. Chaque sprint correspondait à une période définie, durant laquelle un ensemble précis de tâches devait être accompli en fonction des priorités et de la charge estimée.

Pour organiser cette planification de manière claire et visuelle, nous avons utilisé l'outil Trello. Un tableau spécifique au projet a été mis en place, dans lequel les tâches étaient représentées sous forme de cartes et réparties en colonnes selon leur état d'avancement : *À faire*, *En cours* et *Terminé*. Cet outil nous a permis de suivre l'évolution du projet en temps réel, de maintenir un bon rythme de progression et d'assurer une répartition cohérente des efforts tout au long du projet.

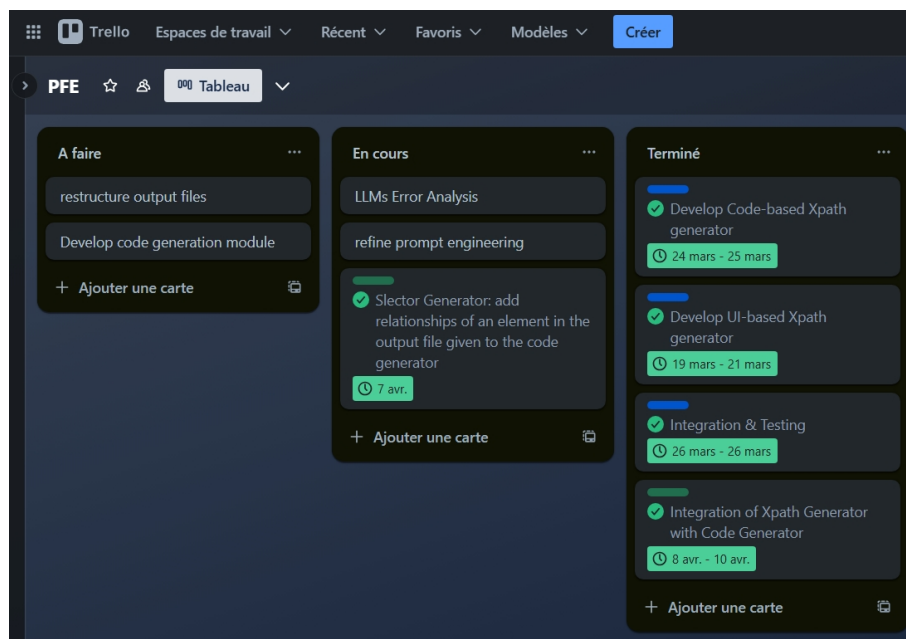


FIGURE 1.3 – Tableau Trello utilisé pour la gestion des tâches par sprint

La planification des tâches a été effectuée avec l'outil Trello, comme présenté dans la figure 1.3.

1.5 Outil de gestion du projet

Au cours de la réalisation de ce projet, plusieurs outils ont été employés afin d’assurer une bonne organisation, une communication fluide, ainsi qu’un suivi efficace des différentes tâches. Le tableau suivant présente les principaux outils utilisés :

TABLE 1.1 – Outils utilisés pour la gestion du projet

Nom	Logo	Description
Microsoft Teams		Outil de collaboration développé par Microsoft, permettant d’échanger par messages instantanés, d’organiser des réunions en ligne, de partager des documents et de travailler en équipe via une interface intégrée à Microsoft 365.
GitHub		Plateforme en ligne basée sur le système de gestion de version Git, utilisée pour héberger du code source, collaborer sur des projets de développement et suivre l’évolution du travail grâce aux fonctionnalités de suivi de versions.
Outlook		Application de gestion de courrier électronique, d’agenda et de tâches, intégrée à la suite Microsoft Office, utilisée pour organiser les communications et planifier les réunions.
OneDrive		Service de stockage cloud proposé par Microsoft, utilisé pour enregistrer, partager et accéder aux fichiers du projet depuis n’importe quel appareil connecté, facilitant la collaboration entre les membres de l’équipe.

1.6 CONCLUSION

Ce chapitre nous a permis de poser les fondations du projet, à travers une présentation claire de l'entreprise Talan, une analyse approfondie des limites de l'automatisation actuelle et une formulation précise de la solution envisagée. La méthodologie de travail choisie, notamment l'approche agile SCRUM, constitue un socle organisationnel solide pour la suite du développement. Le chapitre suivant s'intéresse aux concepts théoriques et technologiques mobilisés pour concevoir cette solution.

Étude conceptuelle et théorique

Sommaire

2.1	INTRODUCTION	14
2.2	Compréhension du métier	14
2.2.1	User Stories	14
2.2.2	Langage Gherkin	15
2.2.3	Documents de spécification	15
2.2.4	Cycle de vie du testing logiciel	15
2.3	Les Besoins fonctionnels et non fonctionnels	16
2.3.1	Analyse et spécifications des besoins	16
2.3.2	Besoins fonctionnels	17
2.3.3	Besoins non fonctionnels	17
2.4	Fondements théoriques	18
2.4.1	Agents intelligents	18
2.4.2	Théorie des agents autonomes	18
2.4.3	Systèmes multi-agents	19
2.4.4	Modèles de Langages de Grandes Tailles (LLM)	19
2.4.5	Traitement du langage naturel (NLP)	20
2.4.6	IA Générative	20
2.4.7	Vision par Ordinateur	21
2.5	Plan de travail	21
2.6	Vision globale du projet	22
2.7	CONCLUSION	23

2.1 INTRODUCTION

Ce chapitre a pour objectif de présenter les concepts clés et les fondements théoriques sur lesquels repose notre projet. Nous y définissons les besoins fonctionnels et non fonctionnels, puis introduisons les notions d’agents intelligents, de modèles de langage (LLM), de traitement du langage naturel (NLP), ainsi que les bases de l’IA générative et de la vision par ordinateur. Enfin, nous décrivons le cycle de vie du testing logiciel et proposons un plan de travail structurant les différentes phases du projet.

2.2 Compréhension du métier

Dans cette section, nous présentons les fondements métier du projet. Avant d’aborder les aspects techniques relatifs à la génération et à l’exécution automatisée des tests, il est indispensable de comprendre le contexte fonctionnel dans lequel ces deux volets prennent place. Cette phase constitue un pilier central dans toute démarche de qualité logicielle, car elle assure une cohérence entre les objectifs métier, les attentes des utilisateurs finaux et les solutions logicielles mises en œuvre.

2.2.1 User Stories

Dans une approche agile, les besoins fonctionnels sont exprimés sous forme de *User Stories*. Celles-ci décrivent une fonctionnalité attendue selon le point de vue de l’utilisateur final, à travers un format standardisé :

En tant que [utilisateur], je veux [fonctionnalité], afin de [objectif métier].

Ce formalisme narratif favorise la clarté des attentes, la communication entre les acteurs du projet (métier, technique, qualité) ainsi que la priorisation des fonctionnalités à développer.

2.2.2 Langage Gherkin

Nous utilisons le langage *Gherkin* pour formaliser les User Stories sous forme de scénarios de test lisibles, compréhensibles par les métiers, mais également directement exploitables techniquement. Gherkin repose sur des mots-clés définis :

- **Given** : définit le contexte ou l'état initial du système.
- **When** : déclenche une action spécifique.
- **Then** : décrit le résultat attendu.
- **And / But** : permet d'ajouter des conditions supplémentaires ou des exceptions.

Cette syntaxe structurée facilite la collaboration entre les développeurs, les testeurs et les représentants métiers. Elle constitue également la base de la génération automatique des scénarios de test dans notre projet.

2.2.3 Documents de spécification

Outre les User Stories, d'autres documents fonctionnels sont exploités afin d'alimenter la base de connaissance du système à tester, notamment :

- Les cahiers des charges fonctionnels.
- Les règles de gestion (business rules).
- Les cas d'usage disponibles dans des fichiers Excel ou via des outils comme Jira.

Ces sources sont essentielles pour la compréhension sémantique des exigences et permettent de générer automatiquement des cas de test cohérents et pertinents.

2.2.4 Cycle de vie du testing logiciel

Le processus de test suit les étapes du *Software Testing Life Cycle (STLC)*, qui structure la démarche qualité en six phases principales :

1. **Analyse des exigences** : identification des besoins métier à couvrir par les tests.

2. **Planification des tests** : définition des stratégies, des ressources et des outils.
3. **Conception des cas de test** : élaboration des scénarios à partir des spécifications.
4. **Préparation de l'environnement** : mise en place des conditions nécessaires à l'exécution.
5. **Exécution des tests** : lancement des cas de test sur le système cible.
6. **Analyse des résultats et clôture** : validation, documentation ²et synthèse.

Dans notre projet, qui repose sur deux volets — la **génération automatique de projets de tests automatisés** et l'**exécution automatisée de ces tests via l'interface utilisateur (UI)** — nous ciblons plus particulièrement les phases de conception et d'exécution. La génération est amorcée dès l'analyse des User Stories, tandis que l'exécution reproduit les interactions réelles avec l'application, garantissant ainsi des tests fidèles au comportement métier attendu.

La figure 2.1 illustre le cycle de vie complet du testing logiciel (STLC).



FIGURE 2.1 – Cycle de vie du testing logiciel (STLC)

2.3 Les Besoins fonctionnels et non fonctionnels

2.3.1 Analyse et spécifications des besoins

Le projet repose sur deux volets distincts : la génération de projets de tests automatisés à partir de scénarios Gherkin, et l'exécution intelligente de ces tests via l'interface utilisateur. Cette section définit les besoins fonctionnels et non fonctionnels propres à chacun de ces volets.

2.3.2 Besoins fonctionnels

2.3.2.1 Génération de projets de tests

- Extraction automatique des scénarios de test depuis des fichiers Gherkin ou des spécifications métiers (Excel, Jira).
- Analyse sémantique des étapes pour identifier les intentions utilisateurs.
- Capture de l'interface utilisateur (DOM et image) et détection des éléments interactifs.
- Appariement entre étapes Gherkin et éléments de l'interface.
- Production de sélecteurs robustes, adaptables à des modifications d'UI.
- Génération de rapports intermédiaires (UI annotée, JSON, logs).

2.3.2.2 Exécution des tests automatisés via l'UI

- Importation des cas de test en Gherkin depuis Jira ou Excel.
- Classification et regroupement intelligent des tests (en fonction de la criticité, des dépendances et de la chronologie).
- Exécution des tests directement sur l'interface utilisateur (via navigateur).
- Gestion des sessions utilisateur et persistance de l'état de navigation.
- Exécution parallèle des tests indépendants.
- Visualisation dynamique des résultats d'exécution via rapports Allure.
- Intégration bidirectionnelle avec des outils de suivi comme Jira.
- Archivage et historisation des suites de tests.

2.3.3 Besoins non fonctionnels

2.3.3.1 Génération de projets de tests

- Précision élevée dans l'analyse sémantique et visuelle.
- Robustesse des sélecteurs générés, même après modifications mineures de l'UI.

- Temps de traitement réduit pour la génération d'un projet complet.
- Réutilisabilité et modularité du code généré.
- Accessibilité de la solution pour des utilisateurs non techniques.

2.3.3.2 Exécution de tests automatisés via l'UI

- Temps d'exécution optimisé grâce à la parallélisation.
- Résilience aux ralentissements et instabilités de l'interface.
- Sécurité des données durant l'exécution (sessions, authentification).
- Utilisation intuitive, sans expertise en LLM ou prompt engineering : le testeur interagit via une interface graphique simple.

2.4 Fondements théoriques

2.4.1 Agents intelligents

La notion d'agent intelligent désigne un système autonome doté de capacités cognitives et d'action lui permettant d'interagir avec un environnement donné. Il perçoit son environnement via des capteurs ou des données, puis prend des décisions et agit via des effecteurs selon des objectifs spécifiques. Les agents peuvent être simples, se comportant de façon réactive, ou complexes, intégrant un modèle interne du monde et des mécanismes d'apprentissage. En synthèse, un agent intelligent est capable de planifier, d'apprendre de son expérience et d'ajuster son comportement pour atteindre ses objectifs sans intervention humaine directe.

2.4.2 Théorie des agents autonomes

La théorie des agents autonomes s'attache à formaliser les principes permettant à un agent de fonctionner de manière indépendante, sans contrôle central. Un agent autonome prend ses propres décisions en fonction de ses perceptions, de ses connaissances internes et de règles prédéfinies, pouvant également apprendre en cours de fonctionnement. Les modèles théoriques d'agents autonomes envisagent différentes architectures (réactives, délibératives, hybrides) pour

structurer le processus de prise de décision. On étudie comment ces agents peuvent gérer des objectifs multiples et s'adapter à un environnement dynamique, souvent en utilisant des techniques d'apprentissage et de planification. L'autonomie implique ainsi une capacité à raisonner sur son propre état, à faire évoluer ses objectifs et à s'auto-gérer pour accomplir des tâches complexes.

2.4.3 Systèmes multi-agents

La notion de système multi-agents désigne un ensemble d'agents autonomes en interaction, coopérant ou concurrençant dans un environnement commun. Chaque agent conserve son autonomie individuelle, mais partage des informations ou se coordonne avec d'autres pour accomplir des tâches collectives. Ces systèmes sont inspirés de comportements sociaux (par exemple des colonies d'insectes ou des sociétés humaines) et modélisent comment un comportement global complexe peut émerger des interactions locales. Ils peuvent être organisés de manière distribuée ou hiérarchique, apportant robustesse et flexibilité au traitement de problèmes volumineux. En conclusion, la théorie des systèmes multi-agents étudie la coordination, la coopération et la résolution distribuée de problèmes entre agents intelligents pour atteindre des objectifs communs.

2.4.4 Modèles de Langages de Grandes Tailles (LLM)

Un modèle de langage de grande taille (Large Language Model, LLM) désigne un réseau de neurones profond entraîné sur d'immenses corpus de données textuelles. Ces modèles, composés de milliards à des centaines de milliards de paramètres, sont capables de comprendre, générer et compléter du texte en langage naturel. Leur entraînement repose souvent sur des méthodes auto-supervisées, où le modèle apprend des régularités statistiques du langage à partir de vastes jeux de données. Les LLM ont montré des performances remarquables dans de nombreuses tâches de traitement automatique du langage, de la génération de texte fluide à la traduction automatique, en passant par la réponse à des questions complexes. Ils constituent l'un des piliers technologiques du NLP moderne (par exemple, les chatbots ou assistants conversationnels avancés).

2.4.5 Traitement du langage naturel (NLP)

Le traitement automatique du langage naturel (NLP) est un sous-domaine de l'intelligence artificielle qui vise à donner aux machines la capacité de comprendre, analyser et générer du langage humain. Il combine des méthodes de linguistique informatique, de statistiques et d'apprentissage automatique pour traiter les données textuelles ou orales. Les tâches typiques du NLP incluent la classification de textes, la reconnaissance d'entités nommées, l'analyse de sentiments, la traduction automatique et la synthèse vocale, entre autres. Les LLM sont devenus essentiels dans ce domaine, car ils fournissent des représentations riches du langage et servent de base aux applications variées du NLP. En conclusion, le NLP cherche à doter les systèmes d'une véritable compétence linguistique, exploitant des modèles comme les LLM pour couvrir de nombreuses applications allant de la recherche d'informations à l'interaction en langage naturel.

2.4.6 IA Générative

L'intelligence artificielle générative regroupe les méthodes de machine learning permettant de créer du contenu original (texte, images, musique, etc.) à partir de données existantes. Ces modèles apprennent la structure sous-jacente d'un ensemble de données et peuvent générer de nouveaux exemples plausibles. Par exemple, en traitant de vastes ensembles d'images, un modèle génératif peut apprendre à produire des images réalistes qui n'ont pas été vues lors de l'entraînement. Parmi les approches phares figurent les réseaux antagonistes génératifs (GAN) et les autoencodeurs variationnels (VAE), qui ont été conçus spécifiquement pour la génération de données nouvelles. Les applications de l'IA générative sont nombreuses : création d'images de haute qualité, génération de texte créatif, augmentation de données pour l'entraînement de modèles, etc.

2.4.7 Vision par Ordinateur

La vision par ordinateur est un domaine de l'intelligence artificielle qui vise à extraire automatiquement de l'information pertinente à partir d'images ou de vidéos. Elle englobe des algorithmes et des modèles permettant de traiter, analyser et interpréter des données visuelles. Les tâches courantes incluent la classification d'images, la détection et la reconnaissance d'objets, la segmentation d'images, la reconstruction 3D et la compréhension de scènes. Ces tâches exploitent majoritairement des réseaux de neurones profonds (tels que les CNN ou les Vision Transformers) capables d'apprendre des représentations abstraites des images. La vision par ordinateur s'applique à de nombreux domaines, de la robotique à la surveillance, en passant par la médecine et les transports intelligents.

2.5 Plan de travail

Dans cette section, nous présentons le plan de travail établi pour la réalisation du projet, réparti sur quatre mois (de février à mai). Le diagramme de Gantt, illustré à la figure 2.2, offre une vue d'ensemble des tâches à exécuter, de leur ordre logique ainsi que de leur durée approximative.

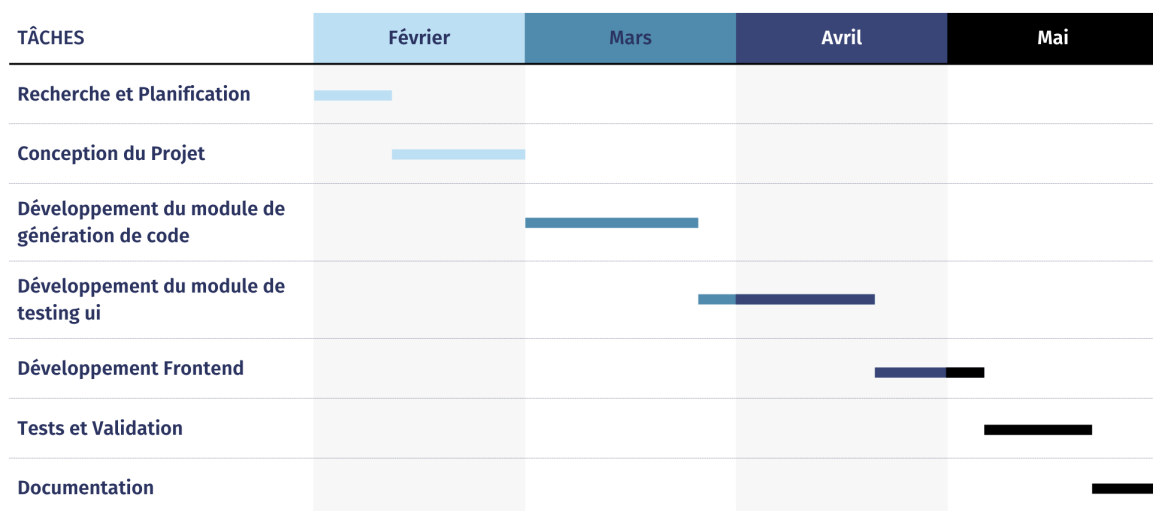


FIGURE 2.2 – Diagramme de Gantt du projet

La figure 2.2 met en évidence sept grandes phases :

- **Recherche et Planification** : Cette phase initiale s'étend sur la première moitié du mois de février. Elle vise à analyser l'état de l'art, cadrer les objectifs du projet et identifier les outils et technologies à utiliser.
- **Conception du Projet** : Elle se déroule durant tout le mois de février. Cette étape inclut la modélisation de l'architecture générale et la définition des modules.
- **Développement du module de génération de code** : Ce module, essentiel pour automatiser la transformation des cas de test en langage Gherkin en scripts de test, est développé sur l'ensemble du mois de mars.
- **Développement du module de testing UI** : Ce module assure l'exécution automatisée des cas de test. Sa mise en œuvre débute mi-mars et se prolonge jusqu'à la fin du mois d'avril.
- **Développement Frontend** : L'interface utilisateur, permettant de gérer les cas de test, est développée principalement durant le mois d'avril, avec des finitions en mai.
- **Tests et Validation** : Cette étape est cruciale pour garantir la fiabilité et la robustesse du système. Elle a eu lieu tout au long du mois de mai, en s'appuyant sur des cas réels.
- **Documentation** : Rédigée en parallèle avec la validation, la documentation finale du projet est complétée au cours de la seconde moitié du mois de mai.

Ce découpage temporel permet une répartition cohérente des efforts et assure une progression incrémentale du projet, en respectant les jalons fixés.

2.6 Vision globale du projet

Avant d'aborder les détails techniques, il convient de présenter la vision d'ensemble du système conçu dans le cadre de ce projet. L'objectif principal est de proposer une solution intelligente et autonome permettant à la fois la **génération de projets de tests automatisés** à partir de scénarios fonctionnels, et **l'exécution de ces tests** via l'interface utilisateur.

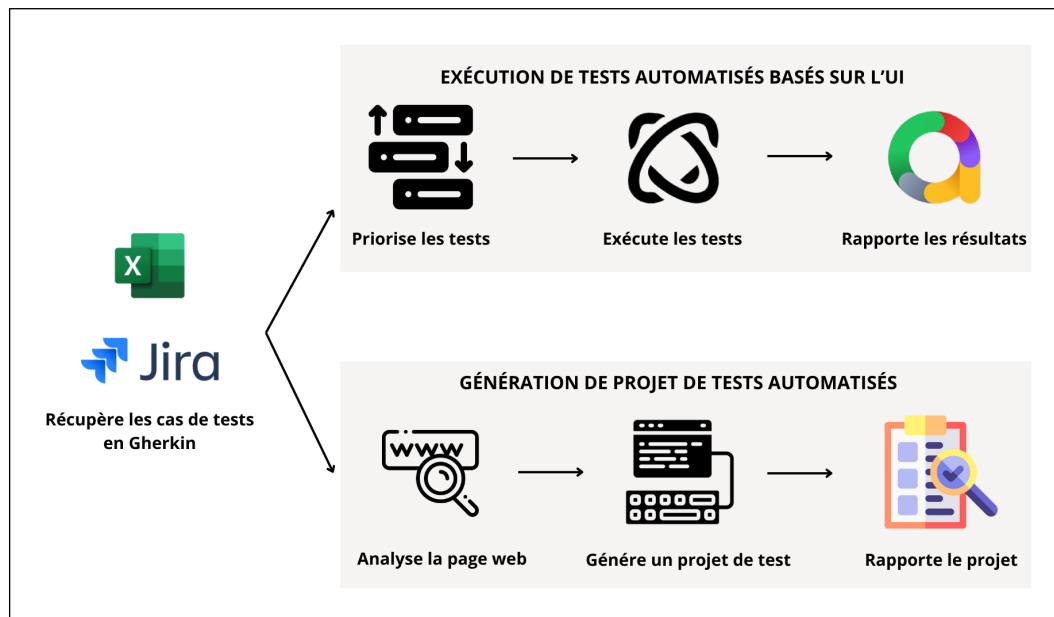


FIGURE 2.3 – Vision globale du projet

La figure 2.3 illustre l'architecture globale du système, qui repose sur deux volets distincts mais complémentaires. Le premier volet est dédié à la génération automatique de scripts de test à partir de scénarios décrits en langage Gherkin, extraits de sources telles que Jira ou des fichiers Excel. Le second volet concerne l'exécution intelligente des cas de test générés, incluant un système de priorisation des scénarios, une interface de contrôle à destination des utilisateurs, ainsi qu'un reporting détaillé permettant de suivre les résultats des campagnes de test.

2.7 CONCLUSION

Nous avons présenté dans ce chapitre les principes théoriques et les choix technologiques qui sous-tendent notre solution. Les notions d'IA générative, d'agents autonomes et de modèles de langage constituent des éléments clés de notre approche. L'analyse des besoins, ainsi que la planification du projet, permettent d'aborder sereinement la phase de conception et de mise en œuvre détaillée, que nous développons dans le chapitre suivant.

Génération de projet de tests automatisés

Sommaire

3.1	INTRODUCTION	25
3.2	Architecture du projet	25
3.3	Fonctionnalités	28
3.4	Outils	30
3.5	Benchmark des outils d'analyse visuelle	30
3.5.1	OmniParser V1 (Microsoft)	31
3.5.2	Pix2Struct (Google)	31
3.5.3	Choix d'OmniParser	32
3.6	Benchmark des LLM testés	33
3.7	Choix des LLM	37
3.8	Gestion de la robustesse des sélecteurs	38
3.9	Structure du code généré	39
3.9.1	Modèle de structure adopté	40
3.9.2	Réutilisabilité du code généré	40
3.9.3	Exemple annoté d'un code test généré	41
3.9.4	Synthèse	41
3.10	CONCLUSION	42

3.1 INTRODUCTION

Ce chapitre décrit la première composante technique de notre solution : la génération automatisée de projets de tests à partir de scénarios en langage Gherkin. Nous y présentons l'architecture adoptée, les fonctionnalités mises en œuvre, les outils utilisés, ainsi que les choix technologiques effectués. Une attention particulière est portée à la robustesse des sélecteurs, à la structure du code généré et à la justification des composants choisis.

3.2 Architecture du projet

Cette section décrit l'architecture technique mise en place pour générer automatiquement des scripts de tests à partir de scénarios Gherkin. Elle détaille les étapes du pipeline, les composants impliqués ainsi que les interactions entre eux. La figure suivante illustre cette architecture fonctionnelle.

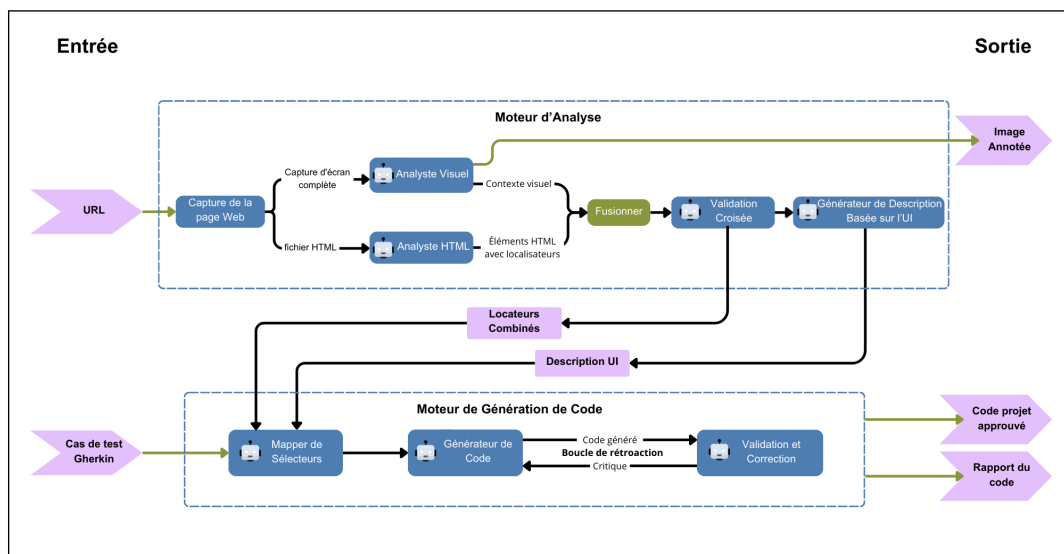


FIGURE 3.1 – Architecture de génération de projets de tests automatisés

L'architecture technique du module de génération est illustrée dans la figure 3.1.

a) Étape 1 – Capture de l'interface utilisateur

Composant : *Capture de page web*

Cette étape constitue la base du processus. Elle s'appuie sur une double collecte : la structure HTML (DOM) et une capture d'écran haute fidélité. Contrairement aux approches classiques, centrées uniquement sur le DOM, cette combinaison permet une analyse conjointe, à la fois structurelle et visuelle, de l'interface.

Outils utilisés : Playwright (principal), Selenium (alternative)

b) Étape 2A – Analyse du DOM (en parallèle)

Composant : *Analyseur HTML*

Exploration de la structure du DOM pour identifier les éléments interactifs (boutons, champs, liens, etc.) et extraire des sélecteurs exploitables (CSS, XPath). Cette analyse est réalisée en parallèle avec l'analyse visuelle, permettant une validation croisée.

Outil utilisé : BeautifulSoup

c) Étape 2B – Analyse visuelle (en parallèle)

Composant : *Analyseur visuel*

À partir de la capture d'écran, cette étape détecte les composants graphiques visibles et en reconnaît les textes associés. Elle permet d'identifier des éléments parfois absents ou mal représentés dans le DOM, ce que les approches classiques ne prennent pas en compte.

Outils utilisés : OmniParser (YOLOv5, PaddleOCR, Florence)

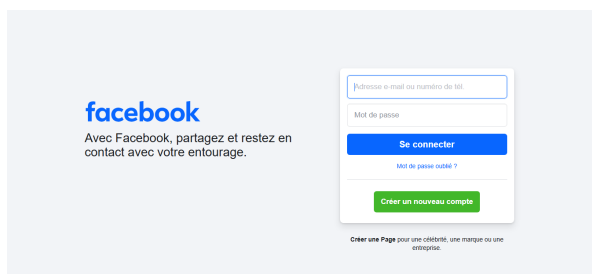


FIGURE 3.2 – Image capturée de l'interface utilisateur



FIGURE 3.3 – Image annotée avec les éléments UI détectés

Les figures 3.2 et 3.3 illustrent un exemple d'extraction automatique des éléments de l'interface utilisateur.

d) Étape 3 – Fusion des informations

Composant : *Moteur de fusion*

Cette étape clé croise les résultats des analyses DOM et visuelle afin de valider, enrichir et désambiguïser les éléments détectés. Cette fusion multimodale constitue une **caractéristique distinctive par rapport aux approches classiques**, généralement limitées à une seule source. Elle permet une représentation robuste et fidèle de l'interface utilisateur.

e) Étape 4 – Description sémantique de l'interface

Composant : *Descripteur d'interface*

Génération d'une description fonctionnelle contextualisée, combinant texte explicatif et image annotée. Cette étape vise à formaliser les intentions d'usage, au-delà de la simple identification d'éléments, comme c'est souvent le cas dans les approches traditionnelles.

Outil utilisé : DeepSeek-V3

f) Étape 5 – Génération du code de test

Composant : *Générateur de code*

Traduction automatique des descriptions (Gherkin) en scripts d'automatisation (Python + Selenium), en intégrant les sélecteurs identifiés. Cette génération est sensible au contexte fonctionnel, contrairement aux méthodes figées des systèmes classiques.

Outils utilisés : Qwen2.5

g) Étape 6 – Validation et relecture du code

Composant : *Critiqueur*

Évaluation automatique de la qualité du script généré, avec la possibilité d’une relecture manuelle. Contrairement aux approches qui négligent cette phase, ce contrôle final assure la conformité et la fiabilité du résultat. L’évaluation repose sur un ensemble de **bonnes pratiques**

reconnues dans l’automatisation avec Selenium, notamment :

- L’utilisation explicite de *waits* dynamiques (`WebDriverWait`) plutôt que de délais fixes.
- La gestion appropriée des exceptions et des états d’attente conditionnelle.
- La clarté des scénarios Gherkin pour assurer la lisibilité et la traçabilité des cas de test.

Outils utilisés : Mistral-7B (Critique), relecture manuelle

h) Conclusion

Ce pipeline se distingue par son approche **modulaire, multimodale et sémantique**, intégrant l’analyse structurelle (DOM) et visuelle dès les premières étapes. La **fusion des sources d’information** constitue une innovation centrale, absente des méthodes traditionnelles. Chaque composant contribue à produire des tests automatisés **traçables, robustes et alignés avec les intentions fonctionnelles** exprimées par l’utilisateur.

3.3 Fonctionnalités

La solution développée automatise l’intégralité du processus de génération de tests fonctionnels avec Selenium, en combinant analyse sémantique, traitement d’interface et production de code. Elle repose sur plusieurs modules complémentaires :

- **Génération automatique de scripts Selenium**

À partir de scénarios en langage *Gherkin* et de l’analyse de l’interface web, la solution génère automatiquement des scripts Python Selenium prêts à l’exécution, sans codage manuel.

– Analyse intelligente de l’interface utilisateur

Le système capture le DOM et une capture d’écran de la page, puis identifie automatiquement les éléments interactifs à tester, en croisant les informations structurelles et visuelles.

– Appariement automatique scénario–UI

Chaque étape fonctionnelle décrite dans les scénarios *Gherkin* est automatiquement associée aux éléments réels de l’interface, assurant une couverture de test cohérente et pertinente.

– Génération de sélecteurs robustes

Les sélecteurs utilisés dans les scripts sont sélectionnés automatiquement en fonction de leur fiabilité (résistance aux changements mineurs de l’UI), garantissant la stabilité des tests.

– Production de rapports détaillés

Un rapport complet est généré à chaque exécution, incluant :

- Les étapes couvertes.
- Les éléments identifiés.
- Les scripts produits.
- Les éventuelles difficultés rencontrées.

– Interface utilisateur conviviale

Une interface web intuitive permet de :

- Charger les scénarios.
- Déclencher les analyses.
- Visualiser les correspondances UI.
- Télécharger les scripts ou rapports.

L’approche est pensée pour rester accessible à des profils non techniques.

Chaque fonctionnalité contribue à rendre l’automatisation des tests plus rapide, fiable et accessible, tout en assurant une parfaite traçabilité entre les exigences métier et les scripts générés.

3.4 Outils

Cette section présente les principaux outils et modules logiciels utilisés dans notre solution. Chaque outil est associé à une fonction spécifique au sein du pipeline, allant de la capture de l'interface à l'analyse visuelle et structurelle. Le tableau ci-dessous en résume les rôles respectifs, ainsi que les étapes du processus auxquelles ils contribuent.

Logo	Outil / Module	Fonction principale	Étapes concernées
	Playwright	Capture de l'interface utilisateur (HTML, screenshots)	Capture
	Selenium	Automatisation de navigateur pour la capture d'écran	Alternative pour la capture
	BeautifulSoup	Analyse de la structure HTML : balises, ID, composants	Analyse HTML
	Florence	Détection multimodale des éléments UI à partir d'images	Analyse visuelle
	YOLOv5	Détection d'objets graphiques (bounding boxes)	Analyse visuelle
	PaddleOCR	Extraction de texte via reconnaissance optique (OCR)	Analyse visuelle

TABLE 3.1 – Outils utilisés pour la capture, l'analyse, la détection visuelle et la gestion des cas de tests

3.5 Benchmark des outils d'analyse visuelle

Dans le domaine de l'analyse visuelle des interfaces utilisateur, deux modèles se distinguent par leurs approches innovantes : **OmniParser V1** de Microsoft et **Pix2Struct** de Google.

3.5.1 OmniParser V1 (Microsoft)

- **Objectif** : Fournir une solution de parsing visuel des interfaces graphiques, permettant l'identification et la description des éléments interactifs à partir des captures d'écran, sans dépendre du code source sous-jacent.
- **Architecture** : Composé de plusieurs modules spécialisés :
 - *Détection d'éléments interactifs* : Utilise un modèle YOLOv5 pour détecter efficacement les composants interactifs sur diverses plateformes.
 - *Module OCR* : Intègre PaddleOCR pour extraire avec précision les textes visibles, y compris ceux situés sous les icônes.
 - *Description d'icônes* : Emploie un modèle Florence finement ajusté pour générer des descriptions fonctionnelles de haute qualité pour les icônes, images, graphiques et tableaux.
- **Performances** : Évalué sur plusieurs benchmarks, OmniParser V1 a démontré des améliorations significatives :
 - *SeeAssign Task* : Amélioration de la précision de GPT-4V de 70.5% à 93.8% grâce à l'incorporation de la sémantique locale.
 - *ScreenSpot Benchmark* : Reconnaissance des widgets texte/icône améliorée de plus de 20% par rapport à GPT-4V.
 - *Mind2Web Benchmark* : Taux de réussite des tâches amélioré de 4.1% à 5.2% en utilisant uniquement des captures d'écran, surpassant les méthodes basées sur le HTML.
 - *AITW Benchmark* : Augmentation de 4.7% des performances de GPT-4V dans la navigation des applications mobiles.

3.5.2 Pix2Struct (Google)

- **Objectif** : Pré-entraînement sur des captures d'écran masquées de pages web pour générer du HTML simplifié, visant à comprendre la structure visuelle des interfaces utilisateur.

- **Architecture** : Modèle encodeur-décodeur basé sur le Vision Transformer (ViT), intégrant des représentations d'entrée à résolution variable et une intégration flexible des entrées linguistiques et visuelles.
- **Performances** : Pix2Struct a atteint des résultats de pointe dans 6 des 9 tâches évaluées sur 4 domaines : documents, illustrations, interfaces utilisateur et images naturelles.
- **Limitations** : Bien que puissant pour comprendre la structure globale des interfaces, Pix2Struct ne fournit pas de détails précis sur les éléments individuels, tels que leurs positions exactes ou leurs types spécifiques, ce qui limite son utilité pour l'extraction de locators précis.

3.5.3 Choix d'OmniParser

Le choix d'**OmniParser** pour l'extraction de locators à partir de captures d'écran d'interfaces utilisateur est motivé par plusieurs facteurs clés :

- **Approche purement visuelle** : Contrairement aux méthodes traditionnelles qui dépendent du code source (HTML, DOM, hiérarchie d'affichage), OmniParser analyse les interfaces graphiques de manière entièrement visuelle, ce qui permet une généralisation efficace sur les plateformes Web, desktop et mobile.
- **Modules spécialisés** : L'intégration d'un détecteur d'éléments interactifs, d'un modèle de description fonctionnelle et d'un module OCR permet une compréhension approfondie des interfaces, facilitant l'identification précise des locators.
- **Performances supérieures** : Les évaluations sur des benchmarks tels que SeeAssign, ScreenSpot, Mind2Web et AITW démontrent des améliorations significatives par rapport aux modèles existants, notamment une augmentation de la précision de 56.8 points sur le benchmark ScreenSpot comparé à GPT-4V seul.
- **Réduction de la latence** : Grâce à l'optimisation de la taille des images traitées, le temps d'inférence a été réduit de 60% par rapport à la version précédente, atteignant en moyenne 0.6 secondes par image sur GPU A100.

- **Compatibilité étendue** : OmniParser est compatible avec divers modèles de langage multimodaux, tels que GPT-4o, DeepSeek R1 et Qwen 2.5VL, offrant une flexibilité d'intégration dans différents environnements.

En résumé, OmniParser représente une solution robuste et efficace pour l'extraction de locators à partir de captures d'écran, surpassant les approches traditionnelles et offrant une généralisation supérieure sur diverses plateformes et interfaces.

3.6 Benchmark des LLM testés

Avant de sélectionner les LLM intégrés au pipeline, nous avons réalisé un benchmark pratique visant à évaluer leurs performances sur des tâches clés du projet. Les modèles ont été testés selon plusieurs configurations de complétion (zero-shot, one-shot, two-shot) sur des jeux d'exemples couvrant :

- Génération de descriptions naturelles d'interfaces graphiques.
- Traduction de scénarios Gherkin en code Selenium Java.
- Mappage de localisateurs (XPath, CSS) à partir de textes.
- Validation et correction automatique de scripts.

Chaque modèle a été évalué selon trois critères principaux :

- **Exactitude syntaxique** : validité du code produit (absence d'erreurs de compilation ou de syntaxe).
- **Adéquation fonctionnelle** : conformité du code généré avec le comportement attendu décrit dans le scénario.
- **Pertinence contextuelle** : capacité du modèle à interpréter correctement les intentions du prompt, en fonction ou non des exemples fournis.

Exemples de prompt utilisés

Afin de mieux cerner l'influence du contexte sur la génération, trois types de prompts ont été expérimentés :

- **Zero-shot** : seule l'instruction et le scénario à traiter sont fournis au modèle (voir Fig. 3.4).
- **One-shot** : un exemple complet d'entrée-sortie précède la requête cible (voir Fig. 3.5).
- **Two-shot** : deux exemples antérieurs sont fournis, renforçant le guidage (voir Fig. 3.6).

La Figure 3.4 montre un prompt en configuration **zero-shot**, où le modèle reçoit uniquement l'instruction et le scénario à traiter sans aucun exemple préalable. Ce cas évalue la capacité brute de généralisation du modèle.

```
messages=[{
  'role': 'user',
  'content': '''Convert the following Gherkin scenario into Selenium WebDriver code using Java.
Include necessary imports, WebDriver setup/teardown, and use explicit waits where needed.
Target elements by ID/XPath when possible.

Scenario: Login with valid credentials
  Given I am on the login page "https://example.com/login"
  When I enter "testuser" in the "username" field
  And I enter "Password123!" in the "password" field
  And I click the "submit" button
  Then I should be redirected to "/dashboard"'''
}],
```

FIGURE 3.4 – Exemple de prompt *zero-shot* pour la génération de code Selenium à partir d'un scénario Gherkin.

La Figure 3.5 illustre un prompt **one-shot**, où un exemple d'entrée-sortie est donné avant le scénario cible. Cela permet au modèle de s'aligner sur un format et un style attendus.

```

messages=[{
  'role': 'user',
  'content': '''Example Input (Gherkin):
Scenario: Add product to cart
  Given I am viewing product "ABC-123" on "https://shop.com/products/ABC-123"
  When I select "Large" from the "size-selector" dropdown
  And I click the "add-to-cart-button"
  Then The cart counter should display "1"

Example Output (Java/Selenium):
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.support.ui.Select;
import org.openqa.selenium.support.ui.WebDriverWait;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.testng.Assert;

public class CartTest {
  public static void main(String[] args) {
    WebDriver driver = new ChromeDriver();
    try {
      // Navigate to product page
      driver.get("https://shop.com/products/ABC-123");

      // Handle size selection
      WebElement sizeDropdown = new WebDriverWait(driver, 10)
        .until(ExpectedConditions.elementToBeClickable(By.id("size-selector")));
      new Select(sizeDropdown).selectByVisibleText("Large");

      // Add to cart
      WebElement addToCart = new WebDriverWait(driver, 10)
        .until(ExpectedConditions.elementToBeClickable(By.id("add-to-cart-button")));
      addToCart.click();

      // Verify cart count
      WebElement cartCounter = new WebDriverWait(driver, 10)
        .until(ExpectedConditions.visibilityOfElementLocated(By.cssSelector(".cart-counter")));
      Assert.assertEquals(cartCounter.getText(), "1", "Cart count mismatch");

    } finally {
      driver.quit();
    }
  }
}

Now convert this new scenario:
Scenario: Search and filter products
  Given I am on the homepage "https://store.example.com"
  When I enter "wireless keyboard" in the "search-box"
  And I select "Price: Low to High" from the "sort-options"
  Then I should see at least 5 results
  And The first result price should be less than $50'''
}],

```

FIGURE 3.5 – Exemple de prompt *one-shot* avec un scénario Gherkin et son code Selenium avant la requête cible.

Enfin, la Figure 3.6 présente un cas **two-shot** : deux exemples complets précèdent le scénario à générer. Cette configuration maximise le contexte pour évaluer la cohérence de génération en environnement guidé.

```

    messages=[{
        'role': 'user',
        'content': '''Example 1 (Search Scenario):

Input (Gherkin):
Scenario: Search for product
    Given I am on the homepage "https://example-store.com"
    When I enter "wireless mouse" in the "search-field"
    And I click the "search-submit" button
    Then I should see at least 3 results
    And All results should contain "mouse"

Output (Java/Selenium):
public class SearchTest {
    public static void main(String[] args) {
        System.setProperty("webdriver.chrome.driver", "chromedriver.exe");
        WebDriver driver = new ChromeDriver();
        try {
            driver.get("https://example-store.com");

            // Search action
            WebElement searchField = new WebDriverWait(driver, 10)
                .until(ExpectedConditions.visibilityOfElementLocated(By.id("search-field")));
            searchField.sendKeys("wireless mouse");
            driver.findElement(By.id("search-submit")).click();

            // Wait for results
            new WebDriverWait(driver, 10)
                .until(ExpectedConditions.titleContains("wireless mouse"));

            // Verify results
            List<WebElement> results = driver.findElements(By.cssSelector(".product-result"));
            Assert.assertTrue(results.size() >= 3, "Minimum results not met");

            for(WebElement result : results) {
                Assert.assertTrue(
                    result.getText().toLowerCase().contains("mouse"),
                    "Irrelevant result found"
                );
            }
        } finally {
            driver.quit();
        }
    }
}

Example 2 (Dropdown Scenario):
Input (Gherkin):
Scenario: Filter by category
    Given I am on the products page "https://example-store.com/products"
    When I select "Electronics" from the "category-filter" dropdown
    Then All results should show "Electronics" category

Output (Java/Selenium):
public class FilterTest {
    public static void main(String[] args) {
        WebDriver driver = new ChromeDriver();
        try {
            driver.get("https://example-store.com/products");

            // Handle native <select> dropdown
            WebElement dropdown = new WebDriverWait(driver, 10)
                .until(ExpectedConditions.visibilityOfElementLocated(By.id("category-filter")));
            new Select(dropdown).selectByVisibleText("Electronics");

            // Wait for filtered results
            new WebDriverWait(driver, 10)
                .until(ExpectedConditions.invisibilityOfElementLocated(By.cssSelector(".loading-spinner")));

            // Verify category tags
            List<WebElement> categories = driver.findElements(By.cssSelector(".product-category"));
            for(WebElement category : categories) {
                Assert.assertEquals(
                    category.getText(),
                    "Electronics",
                    "Incorrect category filtering"
                );
            }
        } finally {
            driver.quit();
        }
    }
}

Now convert this new scenario:
Scenario: Sort and price check
    Given I am viewing search results for "laptop"
    When I select "Price: High to Low" from the "sort-by" dropdown
    Then The first item price should be higher than the last item price'''
    ]],

```

FIGURE 3.6 – Exemple de prompt *two-shot* montrant deux scénarios complets avant le prompt cible.

Résultats comparatifs

Modèle	Gén. Interface	Code Selenium	Validation Code
Claude 3	4.5	4.0	4.5
Mistral-7B	3.5	3.0	3.5
Qwen2.5-Coder	4.0	4.5	3.5
DeepSeek-V3	4.8	3.5	3.0

TABLE 3.2 – Évaluation comparative de plusieurs modèles LLM sur différentes tâches liées à la génération de tests automatisés.

Les résultats présentés sont basés sur des tests internes réalisés dans le cadre de notre projet, ainsi que sur des benchmarks externes tels que HumanEval pour l'évaluation de la génération de code. Ce benchmark a permis d'identifier les forces spécifiques de certains modèles selon les tâches, tout en tenant compte du compromis entre performance, coût et spécialisation dans le choix final.

3.7 Choix des LLM

Le pipeline repose sur des modèles spécialisés, chacun a été sélectionné suite à des expérimentations pratiques (zero-shot, one-shot, two-shot). Ces tests ont permis d'identifier les points forts de chaque LLM dans le pipeline :

- **DeepSeek-V3** : performant pour générer des descriptions naturelles d'interfaces grâce à ses capacités avancées de génération de texte [4].
- **Qwen2.5-Coder-7B** : idéal pour transformer des scénarios Gherkin en code Java Selenium. Il gère bien les imports, les attentes explicites et les assertions [5].
- **Mistral-7B** : léger (quantifié en q4_K_M), il est adapté au mappage de localisateurs avec une bonne compréhension des structures Gherkin [6].

- **Mistral-7B (Critique)** : utile pour valider et corriger automatiquement le code généré, assurant sa fiabilité avant exécution.

Ce choix garantit un bon compromis entre performance, spécialisation et efficacité mémoire.

3.8 Gestion de la robustesse des sélecteurs

La robustesse des *sélecteurs* est un facteur déterminant pour garantir la fiabilité et la maintenabilité des tests automatisés. Afin d'assurer une résistance efficace face aux évolutions de la structure du DOM ou de l'interface utilisateur, plusieurs stratégies ont été mises en œuvre.

a) Validation croisée entre vision et DOM

Une approche d'analyse multi-source a été adoptée, combinant :

- L'extraction des éléments interactifs et de leurs attributs depuis le DOM.
- La détection visuelle des éléments à partir d'une capture d'écran.

Cette validation croisée permet de ne conserver que les éléments réellement visibles et présents à l'écran, limitant ainsi les erreurs liées à des éléments non interactifs ou masqués.

b) Génération de sélecteurs robustes

Lors de la génération des *sélecteurs*, la solution privilégie :

- Les attributs considérés comme stables (id, data-testid, aria-label, etc.).
- Des sélecteurs CSS robustes, en évitant les chemins XPath absolus ou fortement dépendants de la structure hiérarchique.

En l'absence d'attributs fiables, plusieurs caractéristiques sont combinées (type d'élément, texte affiché, contexte DOM) pour produire des *sélecteurs* aussi précis que résilients.

c) Gestion des duplications

En présence de plusieurs éléments similaires (par exemple plusieurs boutons portant le même libellé), le contexte de l'élément est pris en compte :

- Structure parente ou voisine.
- Texte ou libellé associé.

- Ajout conditionnel d'un index ou d'un identifiant contextuel.

d) Correspondance intelligente avec les scénarios de tests

Pour établir une correspondance fiable entre les étapes de test (rédigées Gherkin) et les éléments de l'interface, un mécanisme de correspondance sémantique est mis en œuvre. Celui-ci prend en compte :

- Le type d'élément.
- Le texte affiché ou la description visuelle.
- Les *sélecteurs* générés.

Cette approche permet d'identifier automatiquement l'élément le plus pertinent pour chaque action de test.

e) Résilience et traçabilité

Grâce à cette approche hybride, les tests générés sont conçus pour être résistants aux modifications de l'interface (ajout, renommage ou déplacement d'éléments). En parallèle, tous les résultats d'analyse, les correspondances et les *sélecteurs* générés sont sauvegardés sous forme structurée (*JSON*), assurant une traçabilité complète et facilitant les mises à jour futures.

Ainsi, la solution garantit une automatisation robuste, maintenable et capable de s'adapter aux évolutions rapides de l'interface utilisateur.

3.9 Structure du code généré

Le code de test produit respecte les bonnes pratiques de l'automatisation avec Selenium, en générant des scripts Python structurés et annotés. L'objectif est d'assurer la **lisibilité** et la **robustesse** des tests, tout en facilitant leur maintenance.

3.9.1 Modèle de structure adopté

3.9.1.1 Script de test fonctionnel

Chaque scénario Gherkin est transformé en un script Python autonome, qui :

- Importe les bibliothèques nécessaires (selenium, unittest, etc.).
- Initialise le navigateur et accède à la page cible.
- Exécute les actions utilisateur décrites dans le scénario.
- Utilise les sélecteurs générés automatiquement pour interagir avec l'interface.
- Inclut des assertions explicites pour vérifier les résultats attendus.

3.9.1.2 Assertions explicites

Les tests incluent des **assertions** Python (assert, ou via unittest) pour vérifier :

- La présence ou l'absence d'éléments.
- La validité des messages affichés.
- L'état de l'interface utilisateur après chaque action.

3.9.1.3 Utilisation de waits explicites

Pour garantir la robustesse face aux temps de chargement variables, le code généré utilise des *waits explicites* (WebDriverWait, ExpectedConditions) afin d'attendre la disponibilité ou la visibilité des éléments avant toute interaction.

3.9.2 Réutilisabilité du code généré

Les sélecteurs utilisés dans le script sont annotés en commentaire, ce qui facilite leur compréhension et leur adaptation en cas d'évolution de l'interface.

3.9.3 Exemple annoté d'un code test généré

valid_login_test.py

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

def setup_driver():
    driver = webdriver.Firefox()
    driver.implicitly_wait(10)
    return driver

def teardown_driver(driver):
    driver.quit()

def test_valid_facebook_login(driver):
    driver.get("https://www.facebook.com/")

    wait = WebDriverWait(driver, 10)

    # Given: I am on the Facebook login page
    wait.until(EC.presence_of_element_located((By.CSS_SELECTOR, "#u_0_2_EG")))

    # When: I enter a valid email
    email_field = driver.find_element(By.ID, "email")
    email_field.send_keys("valid_email@example.com")

    # And: I enter a valid password
    password_field = driver.find_element(By.ID, "pass")
    password_field.send_keys("valid_password")

    # And: I click the login button
    login_button = driver.find_element(By.ID, "u_0_5_Wy")
    login_button.click()

    # Then: I should be redirected to the Facebook home page
    wait.until(EC.url_contains("https://www.facebook.com/"))
```

FIGURE 3.7 – Tableau Trello utilisé pour la gestion des tâches par sprint

3.9.4 Synthèse

La structure du code généré permet de produire des tests :

- **Lisibles** et compréhensibles pour les équipes techniques.
- **Robustes** grâce à l'utilisation de waits explicites et de sélecteurs fiables.
- **Faciles à maintenir** et à adapter en cas d'évolution de l'interface.

3.10 CONCLUSION

À travers ce chapitre, nous avons mis en place un système performant de génération de tests automatisés, reposant sur l'analyse combinée du DOM et de l'interface utilisateur. Les outils sélectionnés et la structure modulaire du code généré offrent une solution maintenable, évolutive et adaptée à divers contextes. Le chapitre suivant s'intéresse à l'autre pilier du projet : l'exécution intelligente des tests via l'interface graphique.

Exécution de tests automatisés basés sur l'UI

Sommaire

4.1	INTRODUCTION	44
4.2	Architecture du projet	44
4.3	Fonctionnalités	46
4.4	Outils	47
4.5	Benchmark des outils d'exécution automatisée	47
4.5.1	Comparaison fonctionnelle	48
4.5.2	Synthèse des atouts	48
4.5.3	Choix de Browser-Use	49
4.6	Benchmark des LLM testés	50
4.7	Normalisation des scénarios Gherkin	51
4.7.1	Prétraitement syntaxique	51
4.7.2	Utilisation de modèles d'étapes structurés	51
4.7.3	Agent de reformulation et d'enrichissement	52
4.8	Techniques de <i>Prompt Engineering</i> utilisées	52
4.9	CONCLUSION	53

4.1 INTRODUCTION

Ce chapitre est consacré au deuxième volet technique de notre solution : l'exécution automatisée de scénarios de test via l'interface utilisateur. Nous décrivons l'architecture du système d'exécution, les outils évalués, ainsi que les techniques employées pour structurer les scénarios Gherkin et exploiter efficacement les modèles de langage. Des techniques de prompt engineering viennent renforcer la précision de l'exécution.

4.2 Architecture du projet

L'architecture suivante présente le flux global d'exécution des tests automatisés à partir de scénarios Gherkin ou de documents de spécification. Elle repose sur une chaîne d'agents intelligents qui interviennent dans la synthèse, la priorisation, l'enrichissement et l'exécution des scénarios.

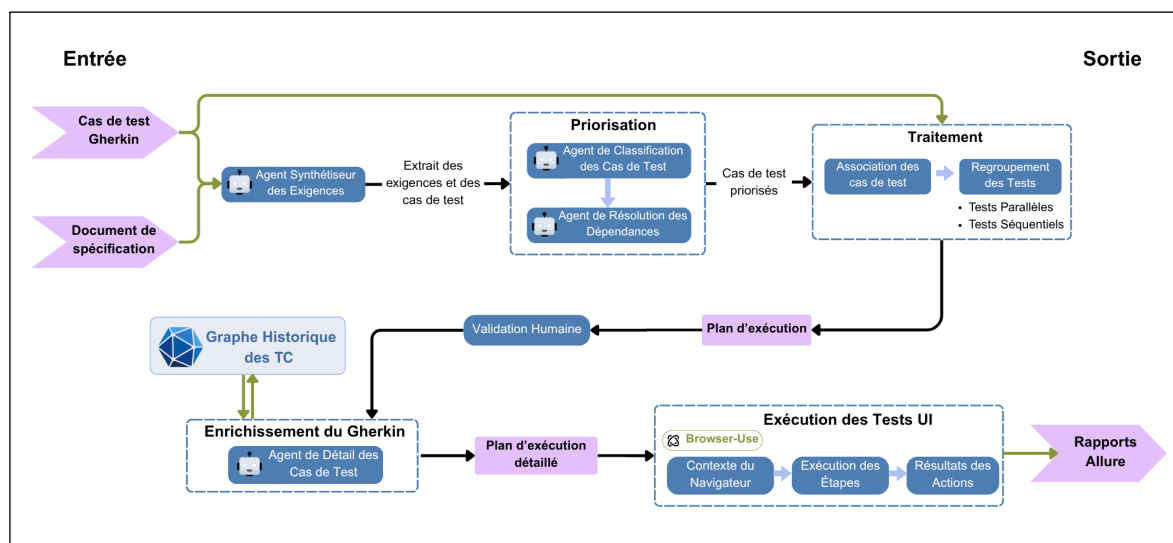


FIGURE 4.1 – Architecture d'exécution de tests automatisés basés sur l'interface utilisateur

Cette architecture se déploie en plusieurs étapes clés, allant de la réception des cas de test jusqu'à la génération de rapports dynamiques via Allure. Le pipeline comprend notamment :

1. **Agent de synthèse des exigences et des cas de test** : cet agent extrait les intentions fonctionnelles clés à partir de documents métier (spécifications et cas de test au format Gherkin). Il traite les entrées par lots.

2. **Module de priorisation assisté par LLM** : à l'aide de modèles de langage, ce module évalue automatiquement l'importance de chaque cas selon des critères métier comme la criticité, la dépendance et la chronologie des tests. Il permet de sélectionner les cas les plus pertinents à exécuter en priorité.
3. **Regroupement par dépendances** : les cas de test sont analysés pour détecter des interdépendances fonctionnelles ou techniques (accès aux mêmes ressources, flux conditionnels, prérequis, etc.). Cela permet de constituer des suites de tests optimisées, organisées pour une exécution parallèle ou séquentielle selon les contraintes détectées.
4. **Agent de détail et d'enrichissement du Gherkin** : ce composant joue un rôle central dans la génération de scénarios de test de qualité et à forte valeur ajoutée. Il intervient après la synthèse initiale pour affiner, compléter et contextualiser les cas de test à l'aide d'une approche enrichie. Son fonctionnement repose notamment sur l'intégration de **GraphRAG (Retrieval-Augmented Generation basée sur graphe)**, qui permet une extraction contextuelle plus fine des éléments fonctionnels. L'agent remplit plusieurs fonctions :
 - **Identification de cas de test manquants** : en analysant les relations implicites et explicites entre les exigences dans un graphe de connaissances, GraphRAG permet de détecter des scénarios non couverts ou partiellement définis.
 - **Enrichissement contextuel des étapes Gherkin** : les étapes des scénarios sont complétées par des données d'entrée réalistes, des variantes de parcours (chemins alternatifs), des scénarios négatifs ou des conditions d'erreur, en s'appuyant sur des nœuds et relations extraits dynamiquement du graphe.
 - **Cohérence et traçabilité améliorées** : en représentant les scénarios et leurs dépendances dans une structure orientée graphe, l'agent assure une meilleure traçabilité entre exigences, règles métier, et cas de test enrichis.

Cette approche garantit une génération de tests plus exhaustive, cohérente et adaptée au contexte métier, tout en réduisant les oublis ou redondances dans la définition des cas de test.

5. **Exécution via Browser-Use** : une fois enrichis et validés, les scénarios Gherkin sont traduits en scripts exécutables interagissant directement avec l'interface utilisateur. Le

moteur Browser-Use assure la navigation, la saisie de données, et les vérifications attendues, de manière robuste et répétable.

6. **Génération de rapports dynamiques** : l'exécution produit des rapports détaillés via Allure. Ces rapports incluent des journaux d'activité, des captures d'écran, des assertions vérifiées et des indicateurs d'avancement. Ils facilitent le diagnostic rapide des anomalies ou écarts de comportement.

4.3 Fonctionnalités

Le système présente plusieurs fonctionnalités avancées :

- **Import flexible des cas de test** : prise en charge des formats Excel et Gherkin pour une intégration rapide.
- **Extraction intelligente des exigences** : identification automatique des besoins à partir de documents métiers.
- **Priorisation automatique** : évaluation des cas de test selon des critères métiers personnalisables (criticité, dépendances et chronologie).
- **Organisation optimisée des tests** : regroupement des tests en suites exécutables de manière parallèle ou séquentielle.
- **Exécution automatisée des tests UI** : interaction directe avec l'interface utilisateur via un moteur d'automatisation intégré.
- **Visualisation des résultats** : rapports dynamiques et détaillés via Allure pour le suivi de l'exécution et des anomalies.
- **Intégration Jira** : synchronisation bidirectionnelle pour une traçabilité complète entre exigences, tests et défauts.
- **Archivage et historisation** : conservation automatique des campagnes de test et de leurs résultats pour analyse ou réexploitation future.
- **Personnalisation avancée** : adaptation fine des critères de priorisation, des formats d'entrée et des livrables générés.

4.4 Outils

Cette section complète la présentation des outils en mettant l'accent sur ceux utilisés lors des phases d'exécution, de gestion et de reporting des tests. Le tableau suivant récapitule les principaux outils retenus, en précisant leur rôle dans le pipeline de test ainsi que les étapes spécifiques auxquelles ils sont associés.

Logo	Outil	Fonction principale	Étapes concernées
	Browser-Use	Exécution automatisée des scénarios de test sur l'interface utilisateur (UI)	Exécution des tests UI
	Jira	Gestion de projet, suivi des tickets, synchronisation des cas de test et traçabilité	Suivi, traçabilité, gestion des anomalies
	Allure	Génération de rapports dynamiques et visuels sur l'exécution des tests	Reporting, visualisation des résultats
	GraphRAG	Exécution de requêtes contextuelles sur des bases de connaissances pour extraire et enrichir les cas de test	Enrichissement des scénarios, extraction intelligente

TABLE 4.1 – Outils principaux utilisés pour l'exécution, la gestion et le reporting des cas de tests

4.5 Benchmark des outils d'exécution automatisée

Dans le cadre de l'automatisation des tests d'interfaces utilisateur (UI), nous comparons deux solutions basées sur l'intelligence artificielle : **Browser-Use** et **CAMEL-AI**.

4.5.1 Comparaison fonctionnelle

Le tableau suivant met en évidence les principales différences des deux solutions selon des critères clés.

Critère	Browser-Use	CAMEL-AI
Architecture	Agent unique dédié à l'automatisation web	Multi-agents pour planification et exécution
Interaction avec l'UI	Contrôle direct via Playwright	Agents spécialisés collaborant sur chaque tâche
Complexité cible	Tâches simples à modérées (clics, formulaires, navigation)	Scénarios complexes nécessitant décomposition et orchestration
Collaboration	Exécution séquentielle, peu ou pas de coordination interne	Échange d'informations et coordination dynamique entre agents
Cas d'usage	Tests UI Web, automatisation ponctuelle, scraping	Automatisation de processus métier complexes, workflows adaptatifs

TABLE 4.2 – Comparaison entre Browser-Use et CAMEL-AI

4.5.2 Synthèse des atouts

- **Browser-Use :**

- Conçu pour des besoins d'automatisation Web ciblés : ouverture de pages, saisies, clics, navigation conditionnelle.
- Simplicité d'intégration dans un pipeline de tests UI : structure monolithique, documentation concise.
- Facilité d'adaptation : code source modulaire permettant d'ajouter rapidement des contrôleurs spécifiques (CAPTCHA simple, surveillance de performance).

- **CAMEL-AI :**

- Conçu pour des cas d'usage complexes : répartition de tâches entre agents, planification automatique.
- Collaboration interne : chaque agent assure une partie du workflow, communication asynchrone.
- Idéal pour l'automatisation de processus métier évolutifs nécessitant une logique de haut niveau.

4.5.3 Choix de Browser-Use

Nous avons retenu **Browser-Use** comme couche d'exécution des tests UI pour les raisons suivantes :

- **Interopérabilité avec GPT-4o** : permet d'envoyer directement des commandes en langage naturel au navigateur pour exécuter chaque étape Gherkin.
- **Fonctionnalités sur mesure** :
 - Ouverture automatique de sites selon le scénario.
 - Actions utilisateur (clics, saisies, navigation dans menus dynamiques).
 - Détection et traitement semi-automatique des éléments bloquants (CAPTCHA simples, pop-ups).
 - Surveillance des temps de réponse et de chargement des éléments critiques.
- **Persistance de session** : l'environnement de navigation reste actif entre les étapes, ce qui permet de simuler des parcours utilisateur complets (authentification, navigation conditionnelle, suivi d'état).
- **Modularité et extensibilité** : architecture open-source facilitant l'ajout de nouveaux contrôleurs spécifiques au contexte applicatif.

Ainsi, Browser-Use offre un contrôle précis et dynamique de l'environnement de test, répondant efficacement aux exigences d'automatisation de la génération, de l'enrichissement et de l'exécution des cas de test UI à partir des spécifications fonctionnelles.

4.6 Benchmark des LLM testés

Pour évaluer les performances des modèles de langage candidats, nous nous sommes appuyés sur plusieurs benchmarks standardisés couvrant un large éventail de tâches : raisonnement, génération de code, compréhension multimodale et performance en contexte multilingue. Le tableau ci-dessous compare les résultats obtenus par GPT-3.5 Turbo, GPT-4 et GPT-4o :

Benchmark	GPT-3.5 Turbo	GPT-4	GPT-4o
MMLU (connaissances générales)	70%	86.6%	88.7%
HumanEval (génération de code)	48.1%	67%	90.2%
MMMU (raisonnement multimodal)	N/A	63.1%	69.1%
MATH (problèmes mathématiques)	34.6%	52.9%	76.6%
GPQA (raisonnement scientifique)	35.7%	47.9%	53.6%

TABLE 4.3 – Comparaison des performances des modèles sur divers benchmarks

Choix du LLM

L'ensemble des agents de l'architecture proposée repose sur GPT-4o d'OpenAI, sélectionné pour les raisons suivantes, directement alignées avec les exigences du projet :

- **Synthèse des exigences et classification** : GPT-4o démontre une compréhension fine du langage naturel, essentielle pour l'extraction automatique des exigences et la catégorisation des cas de test à partir des spécifications.
- **Résolution des dépendances et regroupement** : Ses capacités avancées de raisonnement permettent d'identifier les relations logiques entre les cas de test, facilitant leur ordonnancement (tests séquentiels ou parallèles).
- **Enrichissement des scénarios Gherkin** : La précision syntaxique de GPT-4o garantit la production de scénarios Gherkin cohérents et exploitables.

- **Optimisation du temps et des coûts** : GPT-4o se distingue par sa rapidité et son coût réduit par rapport à GPT-4, tout en maintenant des performances comparables — un atout majeur dans un pipeline de tests automatisés et récurrents.

En résumé, GPT-4o représente le choix optimal pour automatiser l'analyse, l'enrichissement et l'exécution des cas de test UI au sein de notre architecture.

4.7 Normalisation des scénarios Gherkin

Afin de garantir une exécution fiable et cohérente des cas de tests, une phase de normalisation des scénarios en langage Gherkin a été mise en œuvre. Cette étape s'avère essentielle pour réduire l'ambiguïté des inputs et assurer leur bonne interprétation par l'agent d'exécution.

4.7.1 Prétraitement syntaxique

Avant toute exécution, les scénarios Gherkin sont soumis à un processus de prétraitement automatique visant à :

- Éliminer les variations linguistiques inutiles (synonymes, formulations libres).
- Corriger les erreurs de ponctuation ou de structure (mots-clés mal positionnés, phrases incomplètes).
- Découper correctement les étapes en blocs explicites : **Given**, **When**, **Then**, facilitant l'extraction d'intentions.

4.7.2 Utilisation de modèles d'étapes structurés

Pour favoriser la cohérence et la compréhension du contenu par les modèles de langage, un ensemble de **templates standardisés** a été défini. Ces modèles suivent une construction logique autour des étapes clés de Gherkin :

- **Given** : définit l'état initial ou les prérequis (ex. *Given the user is logged in*).
- **When** : décrit l'action déclenchante (ex. *When the user clicks on the login button*).

- **Then** : énonce le résultat attendu (ex. *Then the dashboard should be displayed*).

Ces structures permettent de standardiser les scénarios et de limiter les cas ambigus qui pourraient fausser l'exécution des tests.

4.7.3 Agent de reformulation et d'enrichissement

Un **agent intelligent** a également été développé afin de compléter ou reformuler automatiquement les scénarios incomplets ou peu explicites. Cet agent, basé sur un modèle de langage, est capable de :

- Proposer une version enrichie du scénario initial en déduisant les étapes manquantes ou implicites.
- Détailler automatiquement certaines actions (ex. remplacer *"login"* par *"enter valid credentials and click the login button"*).

Ce processus de normalisation automatisée permet ainsi d'améliorer considérablement la qualité et la précision des scripts générés, tout en facilitant l'interprétation des intentions utilisateur.

4.8 Techniques de *Prompt Engineering* utilisées

Plusieurs techniques de *Prompt Engineering* ont été employées afin d'optimiser l'interaction avec l'agent d'exécution.

- **Zero-Shot Prompting** : Utilisé pour des tâches simples sans fournir d'exemple (ex. extraction directe d'exigences à partir du document de spécification).
- **Few-Shot Prompting** : Fourniture d'exemples pour guider la génération (notamment pour l'enrichissement des scénarios Gherkin).
- **Chain-of-Thought Prompting** : Encouragement du raisonnement pas à pas pour justifier une priorisation ou expliquer un regroupement.
- **Role-Playing** : Positionnement du LLM comme expert métier ou QA pour des décisions plus spécialisées (ex. planification de tests).

- **Template Filling** : Génération automatique de livrables structurés (scénarios Gherkin détaillés) à partir de modèles préétablis.
- **Iterative Prompting** : Raffinement progressif des résultats via des relances successives (ex. amélioration du plan de test initial).
- **Contextual Augmentation** : Ajout de contexte (fonctionnel ou technique) dans le prompt pour des réponses plus précises et pertinentes.

Ces techniques ont été intégrées de manière stratégique dans le pipeline d'automatisation afin de maximiser la pertinence, la robustesse et la réutilisabilité des livrables générés.

4.9 CONCLUSION

Ce chapitre a démontré la faisabilité et l'efficacité d'une exécution entièrement pilotée par l'interface graphique, en simulant fidèlement les actions d'un utilisateur réel. Grâce à l'utilisation de modèles de langage et à une structure de scénario normalisée, l'exécution gagne en fluidité et en fiabilité. Le chapitre suivant présente l'évaluation globale du système mis en place ainsi que l'interface ReactJS développée pour son utilisation.

Résultats et Interface

Sommaire

5.1	Introduction	55
5.2	Évaluation du module de génération de cas de test	55
5.2.1	Temps de génération	55
5.2.2	Robustesse des sélecteurs	55
5.2.3	Alignement scénario-interface	56
5.2.4	Interface de génération	56
5.3	Évaluation du module d'exécution des tests	60
5.3.1	Temps d'exécution	60
5.3.2	Taux de réussite	60
5.3.3	Rapports d'exécution	60
5.3.4	Interface de suivi d'exécution	61
5.4	Apport de l'interface utilisateur ReactJS	63
5.4.1	Objectifs de l'interface	63
5.4.2	Architecture de communication	63
5.4.3	Expérience utilisateur	63
5.5	Conclusion	64

5.1 Introduction

Ce chapitre présente l'évaluation expérimentale de la solution que nous avons développée, en mettant l'accent sur ses deux modules fondamentaux : la génération et l'exécution automatisée de cas de test. Les performances sont analysées à travers des indicateurs concrets, tels que les temps de traitement, la robustesse des sélecteurs, la fidélité d'alignement entre les scénarios et l'interface, ainsi que les taux de réussite. Nous explorons également le rôle central de l'interface utilisateur *ReactJS*, en tant qu'élément de pilotage, de visualisation et d'interprétation des résultats.

5.2 Évaluation du module de génération de cas de test

5.2.1 Temps de génération

Le temps de génération constitue un facteur déterminant dans le processus d'automatisation. À partir d'un fichier Gherkin importé, le système exécute plusieurs étapes : l'analyse linguistique, la capture du DOM, la détection visuelle des éléments et la génération du code de test. L'ensemble de ces opérations est effectué en **127,5 secondes** en moyenne, contre environ **900 secondes** pour un processus manuel. Cette réduction de **86 %** valide la performance de notre architecture pipeline optimisée.

5.2.2 Robustesse des sélecteurs

Pour garantir la maintenabilité des scripts, il est essentiel que les sélecteurs générés soient robustes face aux changements d'interface. Nos tests sur 100 scénarios montrent que **92 %** d'entre eux restent fonctionnels malgré des modifications CSS ou des réorganisations du DOM. Ce résultat est obtenu grâce à la combinaison de données DOM, de reconnaissance visuelle et d'un mécanisme de fallback intelligent basé sur la similarité sémantique et le positionnement.

5.2.3 Alignement scénario–interface

L'un des enjeux majeurs réside dans l'alignement entre les étapes Gherkin décrites et les actions effectivement réalisables sur l'interface utilisateur. Sur 25 scénarios, **88 %** des étapes ont été correctement interprétées et exécutées sans ajustement manuel. Les erreurs résiduelles sont dues à des ambiguïtés de formulation ou à la non-visibilité momentanée de certains composants dans la page.

5.2.4 Interface de génération

L'interface utilisateur permet de visualiser et de contrôler toutes les étapes de génération de manière fluide. Elle se compose de plusieurs écrans successifs illustrés ci-dessous.

UI Test Project Generator

Generate Selenium tests from UI analysis and Gherkin scenarios

Testing Playwright directly...

Direct Playwright test successful!

Step 1: Input Sources

Select Input Method

- ☒ URL
☐ HTML File

Website URL

`https://www.facebook.com`

Gherkin Features Source

- ☒ Existing Directory
☐ Upload Features

Gherkin Features Directory

`fb_features`

Found 2 feature files: ['invalid_login.feature', 'valid_login.feature']

Test Name

`ui_test`

Generate Tests

FIGURE 5.1 – Accueil de l'interface – Importation des fichiers .feature

La figure 5.1 présente l'écran d'accueil, où l'utilisateur peut importer les fichiers '.feature' et saisir l'URL de la page.

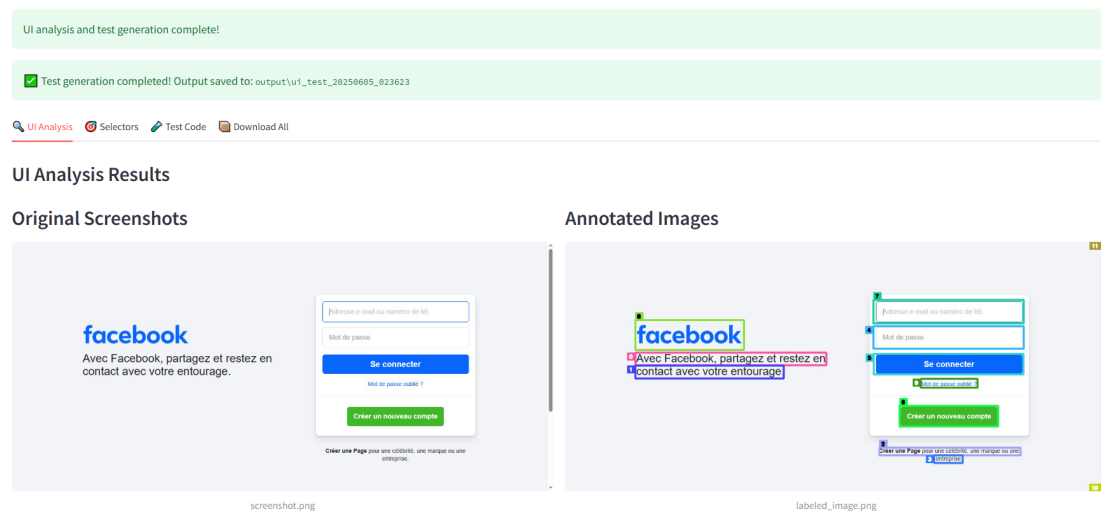


FIGURE 5.2 – Accueil de l’interface – Importation d’un scénario Gherkin

La figure 5.2 présente l’écran d’accueil, où l’utilisateur peut importer un fichier ‘.feature’ issu d’un export Excel.

Selector Mappings

invalid_login.feature valid_login.feature

Feature: invalid_login.feature

Scenario: Login with invalid password (5 steps)

Step	Element Type	Element Text	Locator	Priority
Given I am on the Facebook login page	unknown	Se connecterMot de passe oublié ?Créer un nouveau compte	css=#u_0_2_EG	Medium
When I enter a valid email	input		id=email	High
And I enter an invalid password	input		id=password	High
And I click the login button	button	Se connecter	id=u_0_5_Wy	High
Then I should see an error message	derived		text=I should see an error message	High

FIGURE 5.3 – Visualisation des sélecteurs détectés

La figure 5.3 illustre la détection automatique des composants de l’interface, annotés avec les sélecteurs correspondants.

 UI Analysis  Selectors  **Test Code**  Download All

Generated Test Code

[invalid_login_test.py](#) [valid_login_test.py](#)

valid_login_test.py

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

def setup_driver():
    driver = webdriver.Firefox()
    driver.implicitly_wait(10)
    return driver

def teardown_driver(driver):
    driver.quit()

def test_valid_facebook_login(driver):
    driver.get("https://www.facebook.com/")

    wait = WebDriverWait(driver, 10)

    # Given: I am on the Facebook login page
    wait.until(EC.presence_of_element_located((By.CSS_SELECTOR, "#u_0_2_EG")))

    # When: I enter a valid email
    email_field = driver.find_element(By.ID, "email")
    email_field.send_keys("valid_email@example.com")

    # And: I enter a valid password
    password_field = driver.find_element(By.ID, "pass")
    password_field.send_keys("valid_password")

    # And: I click the login button
    login_button = driver.find_element(By.ID, "u_0_5_Wy")
    login_button.click()

    # Then: I should be redirected to the Facebook home page
    wait.until(EC.url_contains("https://www.facebook.com/"))
```

FIGURE 5.4 – Aperçu du code de test généré

La figure 5.4 présente le code généré automatiquement à partir du scénario.

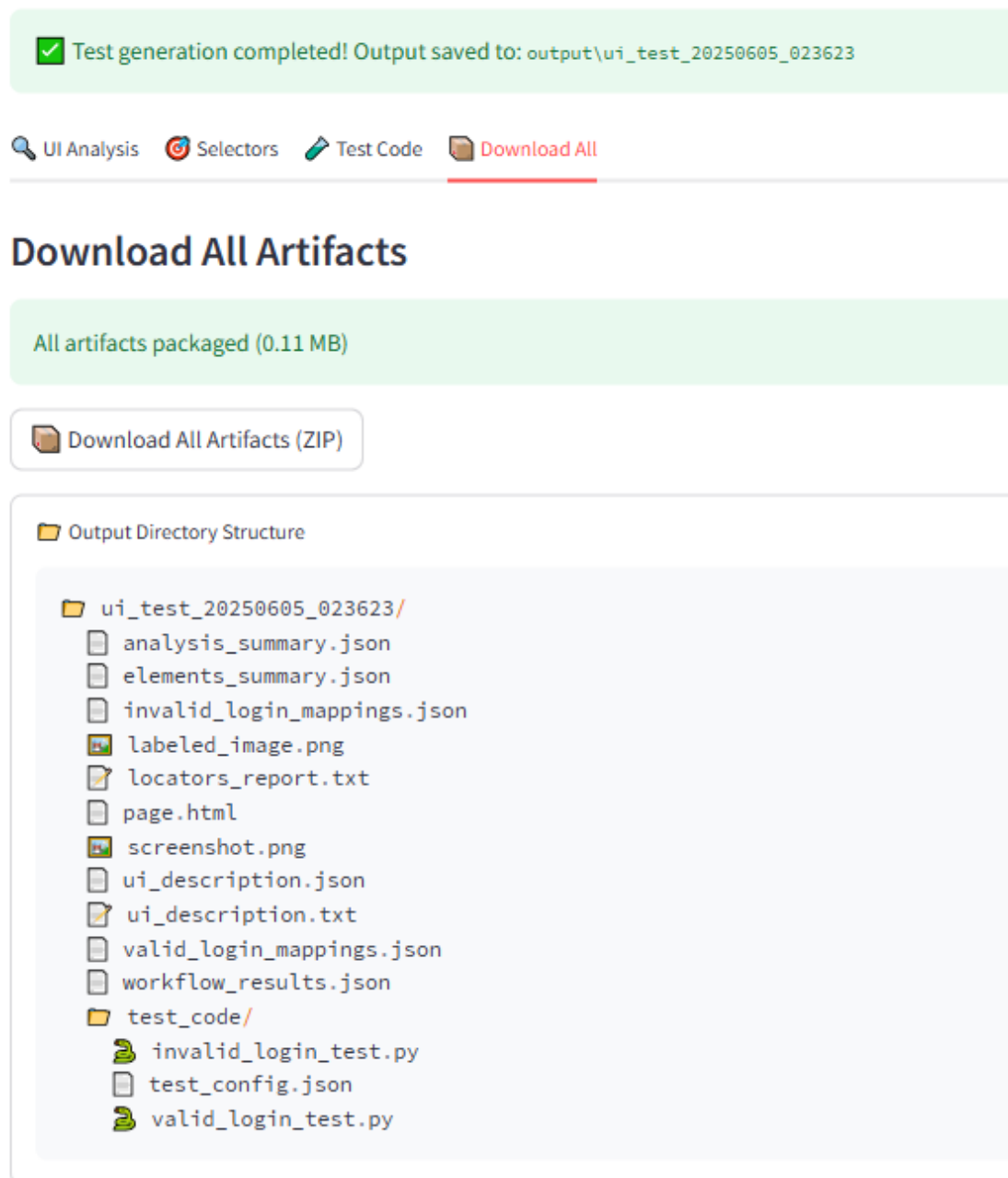


FIGURE 5.5 – Téléchargement du projet de test

La figure 5.5 montre l'étape finale, où l'utilisateur peut télécharger le projet complet prêt à être exécuté.

5.3 Évaluation du module d'exécution des tests

5.3.1 Temps d'exécution

L'exécution constitue un point critique en termes de performance. En mode séquentiel, un scénario complet est exécuté en **125 secondes**. En mode parallèle, ce temps est réduit à **79 secondes**, soit une amélioration de **37 %**. Notre système permet de traiter jusqu'à **8 cas de test en simultané**, grâce à une architecture scalable.

Voici quelques résultats mesurés :

- **Login complexe (2 cas)** : 350 secondes
- **Login simple (2 cas)** : 145 secondes
- **Scénario métier (8 cas)** : 1080 secondes

5.3.2 Taux de réussite

Le taux de réussite mesure la capacité du système à exécuter les cas sans erreur. Sur une campagne de 30 scénarios, **90 %** ont été exécutés avec succès. Les échecs restants proviennent de :

- Délais de chargement excessifs.
- Apparition tardive d'éléments dynamiques.
- Modifications non anticipées du DOM.

5.3.3 Rapports d'exécution

La restitution des résultats est assurée par l'intégration d'Allure Report. L'interface permet à l'utilisateur d'accéder à :

- Une vue d'ensemble des campagnes.
- Les détails des étapes exécutées.
- Les captures d'écran à chaque interaction.
- Les logs et métriques associés.

5.3.4 Interface de suivi d'exécution

L'interface assure une supervision fluide de l'exécution, comme illustré ci-dessous.

Load Gherkin Test Cases & Test Data & App Link

Upload Files Manually

Upload Gherkin Test Cases Excel Files

Choisir des fichiers | Aucun...tionné

Upload Test Data Excel Files

Choisir des fichiers | Aucun...tionné

Application Link

http://example.com

Upload

Upload Jira Data and Test Data Excel Files

Jira Project Key

PROJ

Jira State Field (of Test Cases)

status

Upload Test Data Excel Files

Choisir des fichiers | Aucun...tionné

Application Link

http://example.com

Upload

Run Auto Task

View Allure Report

FIGURE 5.6 – Accueil de la plateforme d'exécution

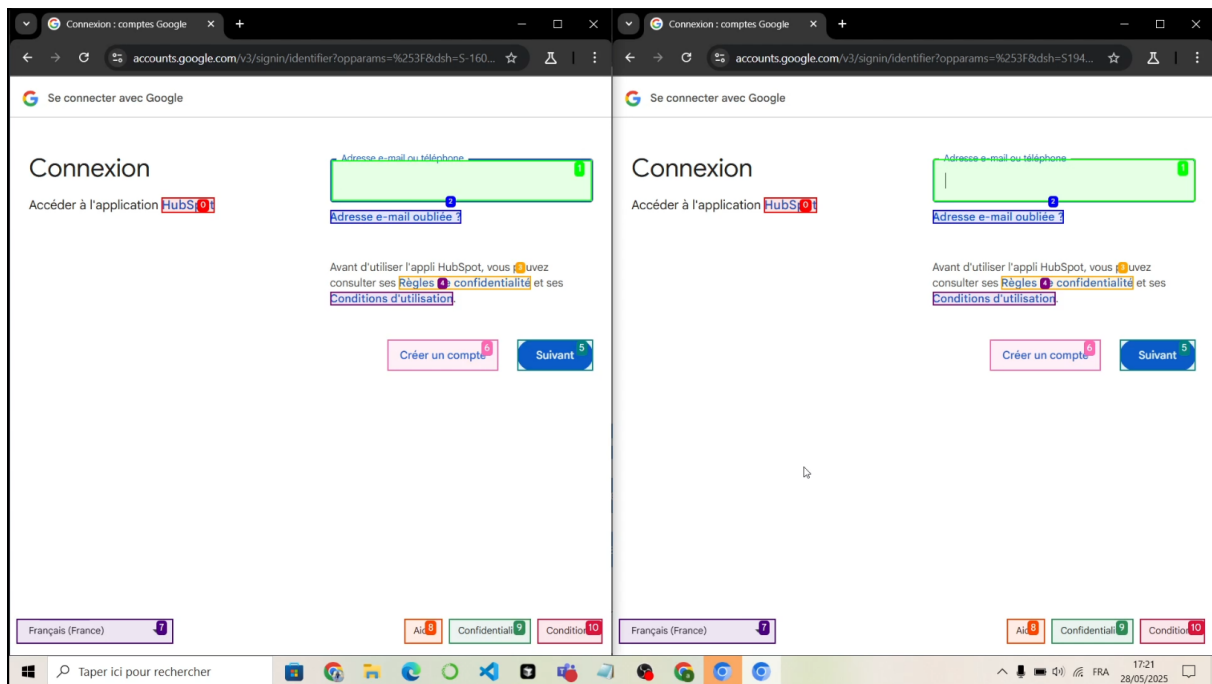


FIGURE 5.7 – Exécution parallèle de cas de test

```
Starting step: Step 3: And I enter "<username>" in the username field
INFO [agent] Step 3
INFO [agent] Eval: Success - The application landing page has been opened.
INFO [agent] Memory: Opened the landing page at https://app-eu1.hubspot.com/login/legacy. 1 out of 15 steps executed.
Next step: Select 'Login with Google' option.
INFO [agent] Next goal: Click the 'Sign in with Google' button to proceed with the login.
INFO [agent] Action 1/1: {"click_element_by_index":{"index":10}}
INFO [controller] Clicked button with index 10:
[DEBUG] agent.state.last_result for step 2: [ActionResult(is_done=False, success=None, extracted_content='Clicked button with index 10: ', error=None, include_in_memory=True)]
Adding action: Action 1: ActionResult
```

FIGURE 5.8 – Étapes de raisonnement de l'agent d'exécution

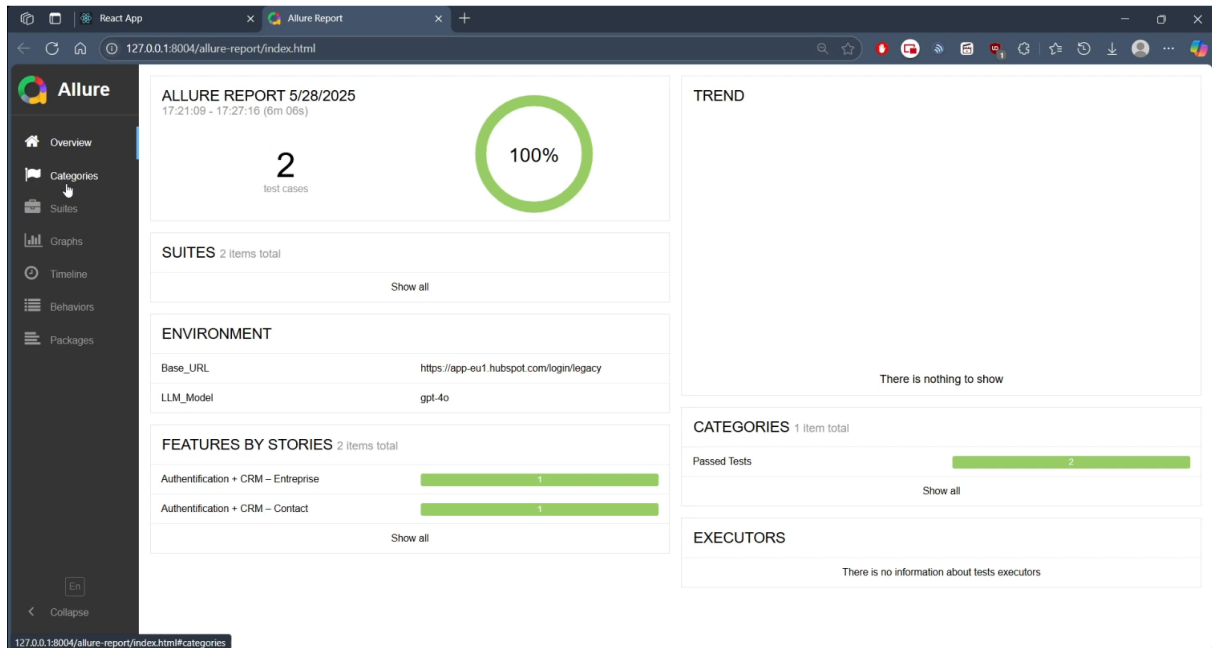


FIGURE 5.9 – Vue d'ensemble du rapport Allure

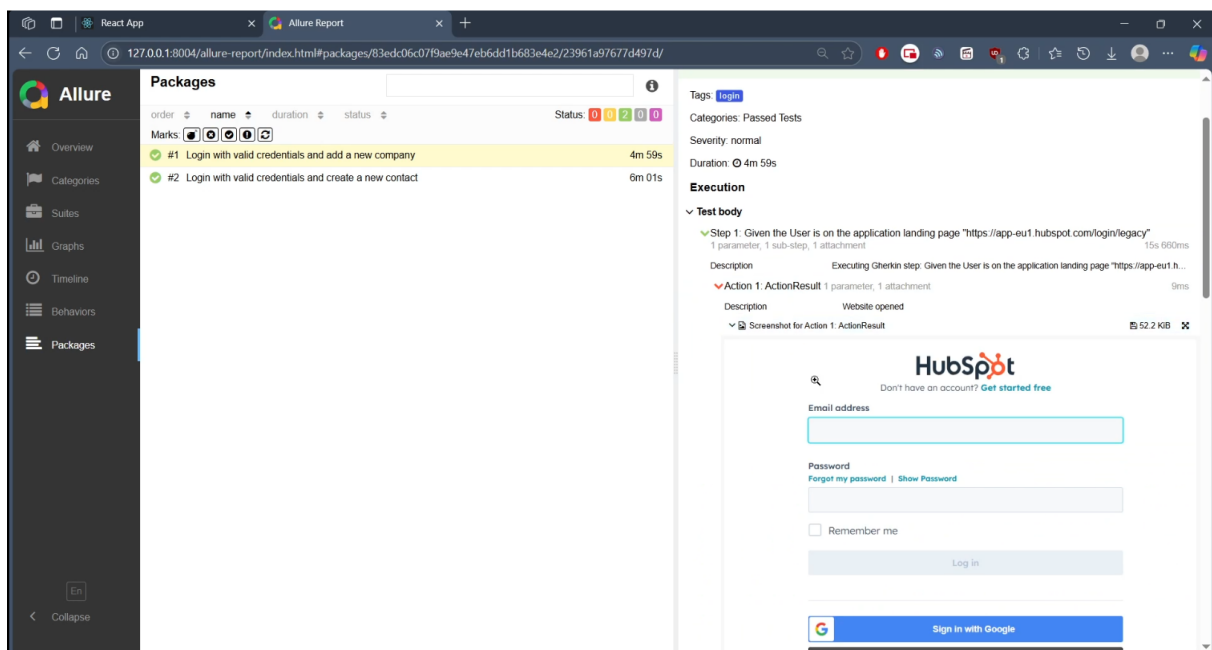


FIGURE 5.10 – Détail du statut d'exécution par cas de test

5.4 Apport de l'interface utilisateur ReactJS

5.4.1 Objectifs de l'interface

L'interface ReactJS a été conçue pour centraliser le pilotage des modules et offrir une expérience fluide et accessible, notamment pour les utilisateurs non techniques. Ses objectifs sont :

- Faciliter l'importation de scénarios Gherkin.
- Permettre le déclenchement des traitements via interface.
- Afficher les résultats en temps réel.
- Réduire la dépendance aux outils en ligne de commande.

5.4.2 Architecture de communication

L'architecture de l'interface repose sur trois composants :

- Un frontend ReactJS pour l'interaction utilisateur.
- Un backend FastAPI pour le traitement de la génération.
- Un service Node.js dédié à l'exécution via Puppeteer.

Les échanges sont réalisés via des APIs REST sécurisées. Cette architecture favorise la modularité et l'évolutivité du système.

5.4.3 Expérience utilisateur

L'interface permet d'exécuter un scénario en trois clics :

- Importer le fichier '.feature',
- Visualiser les annotations et sélecteurs,
- Lancer l'exécution et suivre les résultats.

5.5 Conclusion

Ce chapitre a démontré la performance globale de notre solution, à travers des indicateurs concrets de rapidité, de robustesse et d'efficacité fonctionnelle. L'intégration d'une interface intuitive a contribué à rendre l'outil accessible à un large public, tout en garantissant une traçabilité complète. Ces résultats valident notre approche hybride et posent les bases d'une future extension du système.



CONCLUSION GÉNÉRALE

Ce projet de fin d'études a constitué une opportunité concrète pour explorer les apports de l'intelligence artificielle et de l'automatisation dans le domaine du test logiciel. Dans un contexte où la complexité des applications web et la fréquence des livraisons imposent des cycles de validation toujours plus courts, nous avons conçu une solution innovante capable de générer automatiquement des scripts de test fonctionnel à partir de scénarios Gherkin rédigés en langage naturel.

Notre approche s'est appuyée sur la synergie entre plusieurs technologies complémentaires : l'analyse structurelle du DOM, la vision par ordinateur pour la reconnaissance visuelle des éléments UI, et l'exploitation de modèles de langages de grandes tailles (LLM) pour l'interprétation sémantique des intentions fonctionnelles. Cette combinaison a permis de concevoir un système intelligent, capable de localiser des éléments dans une interface complexe, de les associer à des actions décrites en langage humain, et de produire un code de test robuste et prêt à l'emploi via Selenium.

L'ensemble du pipeline a été implémenté de manière modulaire, facilitant son intégration dans un processus de développement agile. L'interface utilisateur développée offre une expérience fluide et accessible, permettant aux testeurs, même non techniques, de visualiser les résultats, de personnaliser les tests générés, ou de relancer automatiquement leur exécution. Nous avons également mis en œuvre des mécanismes de fallback intelligents pour gérer les cas ambigus ou non résolus, améliorant ainsi la résilience de la solution face à la variabilité des interfaces.

La validation expérimentale a montré une nette amélioration en termes de couverture de test, de résilience aux changements d'UI, et de clarté des scripts produits. Elle a également révélé l'importance de l'interaction entre vision machine et raisonnement linguistique dans les processus de test automatisé de nouvelle génération.

À l'avenir, plusieurs pistes d'amélioration peuvent être explorées : support multi-plateforme (mobile, desktop), adaptation automatique aux évolutions d'interface via apprentissage actif, détection intelligente des régressions fonctionnelles, ou encore intégration avec des outils DevOps pour une automatisation complète du cycle CI/CD. Ces perspectives s'inscrivent dans une démarche de transformation digitale des processus de test, où l'humain reste au centre mais s'appuie sur des outils plus intelligents et collaboratifs.



BIBLIOGRAPHIE

- [1] **Florent Lothon**. Introduction aux méthodes agiles **[en ligne]**. Consulté en 04 mai 2025.
Disponible sur :
<https://agiliste.fr/introduction-methodes-agiles>
- [2] **NUTCACHE**. Les méthodes agiles **[en ligne]**. Consulté en 15 mars 2025. Disponible sur :
<https://www.nutcache.com/fr/blog/les-methodes-agiles/>
- [3] **OpenAI. Operator**. Automatise les tâches en utilisant des agents IA **[en ligne]**. Consulté en 08 mai 2025. Disponible sur :
<https://openai.com/operator>
- [4] **DeepSeek-V3**. Modèle de langage open-source multimodal de haute performance pour la génération de texte et la description visuelle **[en ligne]**. Consulté en 08 mai 2025.
Disponible sur :
<https://huggingface.co/deepseek-ai/DeepSeek-V3-0324>
- [5] **Qwen2.5-Coder-7B-Instruct**. Modèle de génération de code basé sur les scénarios Gherkin **[en ligne]**. Consulté en 19 mai 2025. Disponible sur :
<https://huggingface.co/Qwen/Qwen2.5-Coder-7B-Instruct>
- [6] **Mistral-7B-Instruct-v0.2**. Modèle quantifié q4_K_M utilisé pour les tâches d'instruction et de validation **[en ligne]**. Consulté en 21 mai 2025. Disponible sur :
<https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.2>
- [7] **Schwaber K., Sutherland J.** The Scrum Guide — Le guide officiel Scrum **[en ligne]**. Consulté en 21 mai 2025. Disponible sur :
<https://scrumguides.org/scrum-guide.html>

- [8] **Brown T. et al.** Language Models are Few-Shot Learners [**en ligne**]. Consulté en 23 avril 2025. Disponible sur :
<https://arxiv.org/abs/2005.14165>
- [9] **Microsoft.** Playwright — Framework d’automatisation de navigateurs web [**en ligne**]. Consulté en 06 mars 2025. Disponible sur :
<https://playwright.dev/docs/intro>
- [10] **Selenium Project.** Selenium — Automatisation de navigateurs pour les tests web [**en ligne**]. Consulté en 17 mars 2025. Disponible sur :
<https://www.selenium.dev/documentation/>
- [11] **OmniParser** : A Unified Framework for Text Spotting, Key Information Extraction and Table Recognition. Consulté en 28 mars 2025. Disponible sur :
<https://arxiv.org/abs/2403.19128>
- [12] **Lee K. et al.** Pix2Struct : Screenshot Parsing as Pretraining for Visual Language Understanding [**en ligne**]. Consulté en 29 février 2025. Disponible sur :
<https://arxiv.org/abs/2210.03347>
- [13] **Browser-Use** — Agent IA pour la navigation web automatisée [**en ligne**]. Consulté en 13 février 2025. Disponible sur :
<https://docs.browser-use.com/introduction>

Génération et exécution automatisées de cas de test

Amina CHAABEN

Résumé :

Ce projet présente une solution hybride d'automatisation des tests logiciels. À partir de scénarios Gherkin, le système combine l'analyse DOM et la vision par ordinateur pour identifier les éléments d'interface, puis un modèle de langage génère les scripts Selenium correspondants. L'exécution des tests est assurée par Browser-Use et pilotée via une interface ReactJS, offrant une gestion centralisée des suites et un suivi en temps réel.

Mots clés : Automatisation des tests, Gherkin, DOM, vision par ordinateur, modèle de langage, Selenium, ReactJS.

Abstract :

This project introduces a hybrid solution for software test automation. Starting from Gherkin scenarios, the system combines DOM analysis and computer vision to locate UI elements, and then a language model generates the corresponding Selenium scripts. Test execution is handled by Browser-Use and managed through a ReactJS interface, providing centralized suite management and real-time monitoring.

Key-words : Test automation, Gherkin, DOM, computer vision, language model, Selenium, ReactJS.

تلخيص:

يعرض هذا المشروع حلاً هجيناً لأتمتة اختبارات البرمجيات. ينطلق من سيناريوهات Gherkin، حيث يجمع بين تحليل DOM والرؤية الحاسوبية لتحديد عناصر الواجهة، ثم يُولّد النموذج اللغوي سيناريوهات اختبار باستخدام Selenium. تُنفذ هذه الاختبارات عبر Browser-Use وتُدار من خلال واجهة ReactJS، مما يوفر إدارة مركزية ومتابعة لحظية للتنفيذ.

الكلمات المفاتيح:

أتمتة الاختبارات، Gherkin، الرؤية الحاسوبية، النموذج اللغوي، Selenium، ReactJS.